

BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC KINH TẾ - KỸ THUẬT CÔNG NGHIỆP

KHÓA LUẬN TỐT NGHIỆP

TÊN ĐỀ TÀI KHÓA LUẬN:
**NGHIÊN CỨU, THIẾT KẾ ỨNG DỤNG GIÁM SÁT, PHÂN TÍCH
DỮ LIỆU VÀ CẢNH BÁO THÔNG MINH**

Ngành đào tạo: Công nghệ Kỹ thuật Điện tử - Viễn thông

Mã số ngành: 7510302

Sinh viên thực hiện: Nguyễn Xuân Chúc

Nguyễn Văn Mừng

Người hướng dẫn khóa luận tốt nghiệp:

TS. Vũ Ngọc Dân

Hà Nội - 2026

**BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC KINH TẾ - KỸ THUẬT CÔNG NGHIỆP**

KHÓA LUẬN TỐT NGHIỆP

**TÊN ĐỀ TÀI KHÓA LUẬN:
NGHIÊN CỨU, THIẾT KẾ ỨNG DỤNG GIÁM SÁT, PHÂN TÍCH
DỮ LIỆU VÀ CẢNH BÁO THÔNG MINH**

Ngành đào tạo: Công nghệ Kỹ thuật Điện tử - Viễn thông

Mã số ngành: 7510302

Sinh viên thực hiện: Nguyễn Xuân Chúc

Nguyễn Văn Mừng

Người hướng dẫn khóa luận tốt nghiệp:

TS. Vũ Ngọc Dân

Hà Nội - 2026

LỜI CAM ĐOAN

Nhóm chúng em xin cam đoan những nội dung trong báo cáo này do chúng em tự nghiên cứu và thực hiện dưới sự hướng dẫn của giảng viên hướng dẫn TS. Vũ Ngọc Dân. Các số liệu, hình ảnh và kết quả trình bày trong báo cáo đều do chúng em thực hiện hoặc có trích dẫn rõ ràng nguồn gốc theo đúng quy định. Những số liệu trong các bảng biểu, hình ảnh phục vụ cho việc phân tích, nhận xét, đánh giá do chúng em thu thập từ các nguồn khác nhau.

Chúng em xin chịu hoàn toàn trách nhiệm về tính trung thực và chính xác của nội dung trong báo cáo này. Nếu có bất kỳ vi phạm nào, chúng em xin hoàn toàn chịu trách nhiệm.

Hà Nội, ngày 05 tháng 05 năm 2026

Nhóm sinh viên thực hiện

LỜI CẢM ƠN

Trước tiên, em xin gửi lời cảm ơn chân thành đến Ban Giám hiệu Trường Đại học Kinh tế Kỹ thuật Công nghiệp, cùng Khoa Điện tử và Kỹ thuật máy tính đã tạo điều kiện thuận lợi để chúng em có thể thực hiện và hoàn thành khóa luận với đề tài: *“Nghiên cứu, thiết kế ứng dụng giám sát, phân tích dữ liệu và cảnh báo thông minh”*.

Chúng em xin bày tỏ lòng biết ơn sâu sắc đến thầy Vũ Ngọc Dân, giảng viên của Trường Đại học Kinh tế Kỹ thuật Công nghiệp đã tận tình hướng dẫn, hỗ trợ và đưa ra những ý kiến và kiến thức quý báu của thầy giúp chúng em hoàn thành báo cáo khóa luận này.

Bên cạnh đó, chúng em xin cảm ơn các thầy cô trong khoa Điện tử và Kỹ thuật máy tính của Trường Đại học Kinh tế Kỹ thuật Công nghiệp đã tận tình dạy dỗ và giảng dạy cho chúng em những kiến thức cơ bản đến nâng cao tạo cho chúng em một cơ sở kiến thức vững để hoàn thiện đề tài của mình.

Cuối cùng chúng em xin dành lời cảm ơn đặc biệt đến gia đình và bạn bè, những người đã luôn bên cạnh, ủng hộ và tạo điều kiện tốt nhất để chúng em có thể tập trung học tập và hoàn thành khóa luận này.

Tuy nhiên do giới hạn về thời gian và kinh nghiệm, báo cáo của chúng em không thể tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự đóng góp và phản hồi từ quý thầy cô để đề tài được hoàn thiện hơn.

Chúng em xin chân thành cảm ơn!

Hà Nội, ngày 05 tháng 05 năm 2026

Nhóm sinh viên thực hiện

MỤC LỤC

LỜI CAM ĐOAN	ii
LỜI CẢM ƠN	iii
MỤC LỤC.....	iv
CÁC TỪ VIẾT TẮT	vii
DANH MỤC CÁC BẢNG.....	viii
DANH MỤC CÁC HÌNH.....	ix
PHẦN MỞ ĐẦU	1
CHƯƠNG 1. TỔNG QUAN VỀ ỨNG DỤNG GIÁM SÁT VÀ CẢNH BÁO THÔNG MINH	4
1.1. TẦM QUAN TRỌNG CỦA GIAO DIỆN NGƯỜI DÙNG TRONG PHÒNG CHÁY CHỮA CHÁY (PCCC).....	4
1.1.1. Khái niệm giao diện người dùng trong hệ thống PCCC.....	4
1.1.2. Rút ngắn thời gian phát hiện và phản ứng với sự cố	5
1.1.3. Giảm thiểu sai sót do con người	5
1.1.4. Nâng cao hiệu quả quản lý, bảo trì và phân tích dữ liệu.....	7
1.1.5. Kết luận	8
1.2. XU HƯỚNG ỨNG DỤNG (WEB/APP) TRONG GIÁM SÁT TỪ XA ..	8
1.2.1. Nhu cầu giám sát từ xa trong PCCC	9
1.2.2. Ứng dụng Web trong giám sát hệ thống PCCC.....	9
1.2.3. Mobile App trong cảnh báo và hỗ trợ xử lý.....	10
1.2.4. Các xu hướng công nghệ nổi bật.....	12
1.2.5. Lợi ích và thách thức.....	13
CHƯƠNG 2. CÔNG NGHỆ PHÁT TRIỂN ỨNG DỤNG VÀ XỬ LÝ DỮ LIỆU.....	15
2.1. CÁC FRAMEWORK PHÁT TRIỂN WEB/MOBILE.....	15
2.1.1. Framework phát triển Web	15
2.1.2. Framework phát triển ứng dụng Mobile	17
2.1.3. So sánh và lựa chọn công nghệ phù hợp.....	20
2.2. CÁC THU VIỆN TRỰC QUAN HÓA DỮ LIỆU	21
2.2.1. Tổng quan về trực quan hóa dữ liệu.....	21

2.2.2. Các thư viện trực quan hóa dữ liệu	23
2.2.3. Ứng dụng trong hệ thống giám sát PCCC	24
2.3. CÁC THUẬT TOÁN CẢNH BÁO	25
2.3.1. Nguyên lý phát hiện cháy, khói và rò rỉ gas	25
2.3.2. Thuật toán phát hiện ngưỡng cảnh báo	26
2.3.3. Phân tích dữ liệu và cảnh báo thông minh	27
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG PHẦN MỀM	30
3.1. PHÂN TÍCH USE CASE.....	30
3.1.1. Xác định tác nhân hệ thống.....	30
3.1.2. Biểu đồ Use Case tổng thể	30
3.1.3. Mô tả chi tiết các Use case.....	31
3.2. THIẾT KẾ GIAO DIỆN NGƯỜI DÙNG	33
3.2.1. Nguyên tắc thiết kế UI cho hệ thống giám sát	33
3.2.2. Thiết kế giao diện Dashboard giám sát.....	34
3.2.3. Thiết kế giao diện cảnh báo và thông báo.....	35
3.3. XÂY DỰNG LOGIC CẢNH BÁO THÔNG MINH	37
3.3.1. Thu thập dữ liệu từ cảm biến	37
3.3.2. Xử lý dữ liệu và phát hiện sự cố	40
3.3.3. Cơ chế gửi cảnh báo tới người dùng	41
CHƯƠNG 4. XÂY DỰNG ỨNG DỤNG VÀ ĐÁNH GIÁ TRẢI NGHIỆM	43
4.1. LẬP TRÌNH GIAO DIỆN GIÁM SÁT THỜI GIAN THỰC	43
4.1.1. Kết nối dữ liệu IoT với hệ thống.....	43
4.1.2. Hiển thị trạng thái cảm biến.....	46
4.2. LẬP TRÌNH MODULE VẼ BIỂU ĐỒ LỊCH SỬ.....	49
4.2.1. Lưu trữ dữ liệu cảm biến.....	49
4.2.2. Hiển thị biểu đồ dữ liệu theo thời gian	51
4.3. DEMO TÍNH NĂNG CẢNH BÁO QUA ĐIỆN THOẠI KHI CÓ SỰ CỐ.....	57
4.3.1. Cơ chế gửi thông báo	57
4.3.2. Thử nghiệm hệ thống cảnh báo.....	59

4.4. ĐÁNH GIÁ TRẢI NGHIỆM NGƯỜI DÙNG VÀ ĐỘ TRẺ CỦA THÔNG BÁO	62
4.4.1. Phương pháp đánh giá.....	62
4.4.2. Kết quả thử nghiệm.....	64
4.4.3. Phân tích và nhận xét	66
KẾT LUẬN VÀ ĐỀ NGHỊ.....	69
TÀI LIỆU THAM KHẢO.....	70

CÁC TỪ VIẾT TẮT

Từ viết tắt	Mô tả chi tiết	Giải thích
AI	Artificial Intelligence	Trí tuệ nhân tạo
App	Application	Ứng dụng phần mềm
BMS	Building Management System	Hệ thống quản lý tòa nhà
Cloud	Cloud Computing	Điện toán đám mây
DAO	Data Access Object	Lớp truy xuất dữ liệu
IoT	Internet of Things	Internet vạn vật
Mobile App	Mobile Application	Phần mềm chạy trên thiết bị di động
MQTT	Message Queuing Telemetry Transport	Giao thức truyền thông nhẹ trong IoT
MVVM	Model - View - ViewModel	Dữ liệu - Giao diện - Xử lý trung gian
PCCC	Fire Protection and Fighting	Phòng cháy chữa cháy
SCADA	Supervisory Control and Data Acquisition	Hệ thống giám sát và thu thập dữ liệu
SPA	Single Page Application	Ứng dụng một trang web
UI	User Interface	Giao diện người dùng
Web	Website/Web Application	Ứng dụng web
Wi-Fi	Wireless Fidelity	Kết nối không dây
5G	Fifth Generation	Công nghệ mạng di động thế hệ thứ 5

DANH MỤC CÁC BẢNG

Bảng 2.1. So sánh nền tảng Web và nền tảng Mobile	20
Bảng 2.2. So sánh giữa Native Android (Kotlin) và Cross-platform	21
Bảng 4.1. Quy tắc phân loại trạng thái cảm biến theo ngưỡng dữ liệu	47
Bảng 4.2. Bảng đo độ trễ thông báo.....	63
Bảng 4.3. Tiêu chí đánh giá trải nghiệm người dùng	64
Bảng 4.4. Kết quả tổng hợp độ trễ thông báo	64
Bảng 4.5. Kết quả khả năng gửi thông báo	65
Bảng 4.6. Kết quả đánh giá trải nghiệm người dùng	65

DANH MỤC CÁC HÌNH

Hình 1.1. Công nghệ IoT.....	12
Hình 2.1. Framework	15
Hình 2.2. Một số Web Framework phổ biến.....	16
Hình 2.3. Các Framework phát triển Web	17
Hình 2.4. Giao diện môi trường phát triển Android Studio	18
Hình 2.5. Kotlin - Ngôn ngữ lập trình hiện đại sử dụng trong ứng dụng Mobile Android	19
Hình 2.6. Ví dụ về Dashboard.....	22
Hình 2.7. Sơ đồ khối phát hiện cháy, khói và rò rỉ gas.....	25
Hình 3.1. Biểu đồ Use Case tổng thể của hệ thống PCCC	30
Hình 3.2. Giao diện chính của ứng dụng	34
Hình 3.3. Giao diện lịch sử giám sát.....	36
Hình 3.4. Cảm biến nhiệt độ DHT22	37
Hình 3.5. Cảm biến khí gas MQ-2	37
Hình 3.6. Cảm biến khói MP-2	38
Hình 3.7. Cảm biến lửa	38
Hình 3.8. Lưu đồ xử lý dữ liệu và phát hiện sự cố	40
Hình 3.9. Sơ đồ cơ chế cảnh báo tới người dùng.....	41
Hình 4.1. Sơ đồ luồng dữ liệu IoT	44
Hình 4.2. Giao diện hiển thị trạng thái kết nối MQTT	46
Hình 4.3. Giao diện hiển thị trạng thái khi đã kết nối với MQTT	46
Hình 4.4. Giao diện hiển thị các trạng thái của các phòng khi gặp sự cố.....	47
Hình 4.5. Giao diện hiển thị thông số cảm biến của một khu vực.....	48
Hình 4.6. Giao diện biểu đồ lịch sử dạng cột.....	52
Hình 4.7. Giao diện lọc dữ liệu lịch sử theo khung thời gian trên ứng dụng .	53
Hình 4.8. Giao diện lịch sử khi không có dữ liệu	54
Hình 4.9. Giao diện lịch sử khi truy xuất ra dữ liệu	54
Hình 4.10. Biểu đồ tổng quan bất thường của các phòng.....	56
Hình 4.11. Giao diện ứng dụng khi phòng bếp chuyển mức cảnh báo.....	60

Hình 4.12. Thông báo cảnh báo hiển thị khi không mở trong App	61
Hình 4.13. Giao diện ứng dụng khi phòng bếp chuyển mức nguy hiểm	61

PHẦN MỞ ĐẦU

1. ĐẶT VẤN ĐỀ, TẦM QUAN TRỌNG VÀ LÝ DO CHỌN ĐỀ TÀI

Bối cảnh nghiên cứu:

Trong bối cảnh cách mạng công nghiệp 4.0 đang diễn ra mạnh mẽ, các công nghệ như Internet of Things (IoT), điện toán đám mây và trí tuệ nhân tạo (AI) đang được ứng dụng rộng rãi trong nhiều lĩnh vực đời sống và sản xuất. Đặc biệt, trong lĩnh vực an toàn và phòng cháy chữa cháy, các công nghệ hiện đại giúp nâng cao khả năng giám sát, phát hiện sớm và xử lý kịp thời các sự cố như cháy nổ hoặc rò rỉ khí gas.

Hạn chế của hệ thống truyền thống:

Các hệ thống PCCC truyền thống hoạt động cục bộ tại từng công trình, phụ thuộc vào sự trung tâm và yêu cầu người vận hành có mặt trực tiếp. Những hạn chế chính bao gồm:

- Khó giám sát từ xa.
- Chậm trễ trong tiếp nhận thông tin cảnh báo.
- Thiếu khả năng phân tích dữ liệu để dự đoán rủi ro.
- Ngược lại, IoT cho phép các cảm biến (khói, nhiệt độ, khí gas) kết nối mạng, truyền dữ liệu thời gian thực về máy chủ, xây dựng hệ thống giám sát thông minh.

Vai trò của Web và Mobile App:

Sự phát triển của nền tảng Web và ứng dụng di động cho phép giám sát hệ thống PCCC mọi lúc, mọi nơi. Người dùng có thể:

- Theo dõi trạng thái hệ thống qua giao diện trực quan.
- Nhận cảnh báo tức thì khi có sự cố.
- Thực hiện điều khiển từ xa.
- Kết hợp phân tích dữ liệu, hệ thống không chỉ cảnh báo mà còn dự đoán rủi ro và hỗ trợ quyết định.

Lý do chọn đề tài:

Xuất phát từ những yêu cầu thực tiễn đó, đề tài “Nghiên cứu, thiết kế ứng dụng giám sát, phân tích dữ liệu và cảnh báo thông minh” được lựa chọn nhằm xây dựng một giải pháp giám sát và cảnh báo hiệu quả, góp phần nâng cao mức độ an toàn cho con người và tài sản. Trong khuôn khổ đề tổng thể: “Nghiên cứu, thiết kế hệ thống cảnh báo và rò rỉ gas thông minh dựa trên nền tảng IoT”

nội dung nghiên cứu này tập trung vào việc nghiên cứu, thiết kế ứng dụng giám sát, phân tích dữ liệu và cảnh báo thông minh, đóng vai trò là cầu nối giữa hệ thống IoT và người dùng.

2. MỤC TIÊU NGHIÊN CỨU

Mục tiêu chính của nghiên cứu là xây dựng một ứng dụng có khả năng thu thập dữ liệu từ hệ thống cảm biến, hiển thị thông tin một cách trực quan, hỗ trợ phân tích dữ liệu và đưa ra cảnh báo kịp thời khi phát hiện các dấu hiệu bất thường. Ứng dụng được phát triển trên nền tảng Mobile, đảm nhiệm vai trò cảnh báo nhanh và hỗ trợ người dùng trong các tình huống khẩn cấp.

3. ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU

Đối tượng nghiên cứu của đề tài là hệ thống giám sát và cảnh báo thông minh trong phòng cháy chữa cháy dựa trên nền tảng IoT, tập trung dữ liệu từ cảm biến khói, nhiệt độ và khí gas, hệ thống thu thập và truyền dữ liệu. Và cũng như ứng dụng phần mềm mobile phục vụ giám sát, phân tích và cảnh báo cho người dùng.

Phạm vi nghiên cứu của đề tài bao gồm việc tìm hiểu các công nghệ liên quan như IoT, các giao thức truyền dữ liệu như MQTT, các framework phát triển Web và Mobile, cũng như các thư viện trực quan hóa dữ liệu. Đồng thời, nghiên cứu cũng tiến hành xây dựng hệ thống phần mềm, thiết kế giao diện người dùng, triển khai các thuật toán cảnh báo và thực nghiệm trên mô hình thực tế để đánh giá hiệu quả hoạt động của hệ thống.

4. PHƯƠNG PHÁP NGHIÊN CỨU

Phương pháp nghiên cứu được áp dụng trong đề tài bao gồm:

- Thu thập và phân tích tài liệu chuyên ngành.
- Thiết kế hệ thống phần mềm.
- Xây dựng mô hình thử nghiệm.
- Lập trình ứng dụng.
- Đánh giá kết quả thông qua thực nghiệm.
- Nghiên cứu còn kết hợp so sánh và phân tích các công nghệ nhằm lựa chọn giải pháp phù hợp với yêu cầu của hệ thống.

5. CẤU TRÚC BÁO CÁO

Báo cáo khóa luận được tổ chức thành các chương, mỗi chương sẽ được trình bày một khía cạnh cụ thể của nghiên cứu về thiết kế ứng dụng giám sát,

phân tích dữ liệu và cảnh báo thông minh. Dưới đây là giới thiệu sơ lược về cấu trúc của báo cáo:

- Chương 1: Tổng quan về ứng dụng giám sát và cảnh báo thông minh.

Chương này trình bày tổng quan về hệ thống giám sát và cảnh báo thông minh trong lĩnh vực PCCC, trong đó tập trung phân tích vai trò của giao diện người dùng trong việc hỗ trợ vận hành và xử lý sự cố. Đồng thời, chương này cũng làm rõ xu hướng ứng dụng Web/Mobile trong giám sát từ xa và những lợi ích mà công nghệ IoT mang lại.

- Chương 2: Công nghệ phát triển ứng dụng và xử lý dữ liệu.

Giới thiệu các công nghệ được sử dụng để phát triển hệ thống, bao gồm các framework Web và Mobile, thư viện trực quan hóa dữ liệu và các phương pháp xử lý dữ liệu. Bên cạnh đó, chương cũng trình bày các thuật toán phát hiện và cảnh báo cháy, khói và rò rỉ gas dựa trên dữ liệu cảm biến.

- Chương 3: Phân tích và thiết kế hệ thống phần mềm.

Chương này tập trung vào việc phân tích yêu cầu và thiết kế hệ thống phần mềm, bao gồm xác định tác nhân, xây dựng biểu đồ Use Case và mô tả chi tiết các chức năng. Ngoài ra, chương này còn trình bày thiết kế giao diện người dùng và xây dựng logic xử lý cảnh báo thông minh.

- Chương 4: Xây dựng ứng dụng và đánh giá trải nghiệm.

Trình bày quá trình xây dựng và triển khai ứng dụng thực tế, bao gồm kết nối hệ thống IoT, hiển thị dữ liệu thời gian thực và xây dựng chức năng cảnh báo. Cuối cùng, chương thực hiện đánh giá trải nghiệm người dùng và hiệu năng hệ thống thông qua các thử nghiệm thực tế.

CHƯƠNG 1. TỔNG QUAN VỀ ỨNG DỤNG GIÁM SÁT VÀ CẢNH BÁO THÔNG MINH

1.1. TẦM QUAN TRỌNG CỦA GIAO DIỆN NGƯỜI DÙNG TRONG PHÒNG CHÁY CHỮA CHÁY (PCCC)

Trong các hệ thống phòng cháy chữa cháy (PCCC) truyền thống, trọng tâm thường được đặt vào phần cứng như đầu báo cháy, tủ trung tâm, hệ thống chữa cháy tự động,... Tuy nhiên, khi xu hướng số hóa và giám sát tập trung phát triển, giao diện người dùng ngày càng trở nên then chốt quyết định hiệu quả vận hành hệ thống PCCC.

1.1.1. Khái niệm giao diện người dùng trong hệ thống PCCC

Giao diện người dùng trong hệ thống phòng cháy chữa cháy (PCCC) là tập hợp các thành phần cho phép con người quan sát, theo dõi và tương tác với hệ thống ứng dụng. Thông qua giao diện này, người vận hành có thể kiểm tra trạng thái hoạt động của hệ thống, nhận cảnh báo khi xảy ra sự cố, cũng như thực hiện các thao tác điều khiển cần thiết.

Trong hệ thống PCCC hiện nay, giao diện người dùng thường bao gồm các thành phần chính sau:

- Màn hình hiển thị của tủ trung tâm báo cháy: Đây là nơi hiển thị các thông tin quan trọng như trạng thái hoạt động của hệ thống, vị trí xảy ra cháy hoặc rò rỉ gas, các cảnh báo lỗi và các thông báo liên quan.
- Phần mềm giám sát trên máy tính: Hệ thống có thể được tích hợp với phần mềm giám sát chạy trên máy tính dưới dạng web hoặc ứng dụng desktop, giúp người quản lý theo dõi dữ liệu, phân tích tình trạng hệ thống và nhận cảnh báo từ xa.
- Ứng dụng di động: Cho phép người dùng nhận thông báo cảnh báo và theo dõi trạng thái hệ thống thông qua điện thoại thông minh, giúp nâng cao khả năng giám sát và phản ứng nhanh khi xảy ra sự cố.
- Các thiết bị điều khiển trực tiếp: Bao gồm bảng điều khiển, đèn báo trạng thái, còi báo động và nút nhấn khẩn cấp. Những thiết bị này giúp người dùng có thể thao tác nhanh chóng khi cần kích hoạt cảnh báo hoặc xử lý tình huống khẩn cấp.

1.1.2. Rút ngắn thời gian phát hiện và phản ứng với sự cố

Trong các sự cố cháy nổ, thời gian phản ứng đóng vai trò vô cùng quan trọng. Chỉ trong vài giây, đám cháy có thể lan rộng và gây ra những thiệt hại nghiêm trọng về người và tài sản. Vì vậy, giao diện người dùng trong hệ thống phòng cháy chữa cháy cần được thiết kế trực quan, dễ sử dụng và cung cấp thông tin nhanh chóng để hỗ trợ người vận hành đưa ra quyết định kịp thời.

Thứ nhất, giao diện người dùng giúp truyền đạt thông tin nhanh và rõ ràng. Thông tin trạng thái của hệ thống thường được phân biệt bằng màu sắc trực quan nhằm giúp người dùng nhận biết tình trạng ngay lập tức mà không cần đọc quá nhiều thông tin chi tiết. Cụ thể:

- Màu xanh: Hệ thống hoạt động bình thường.
- Màu vàng: Cảnh báo hoặc xuất hiện lỗi kỹ thuật cần kiểm tra.
- Màu đỏ: Có cháy hoặc nguy cơ cháy nghiêm trọng cần xử lý khẩn cấp.

Giao diện còn hiển thị vị trí xảy ra cháy trên sơ đồ tòa nhà theo thời gian thực, bao gồm các thông tin như tầng, khu vực và phòng cụ thể. Điều này giúp lực lượng trực vận hành hoặc lực lượng cứu hỏa nhanh chóng xác định chính xác vị trí sự cố và triển khai phương án xử lý phù hợp.

Thứ hai, giao diện người dùng giúp giảm số bước thao tác trong quá trình xử lý sự cố. Một giao diện được thiết kế hợp lý sẽ cho phép người vận hành thực hiện các thao tác quan trọng ngay từ màn hình chính mà không cần truy cập qua nhiều menu phức tạp. Cụ thể, từ giao diện chính, người trực hệ thống có thể:

- Xác nhận tín hiệu báo cháy.
- Xem chi tiết thông tin cảm biến hoặc khu vực phát hiện cháy.
- Kích hoạt hoặc tạm ngắt các cơ chế liên quan như chuông báo cháy, loa thông báo, cửa từ, hệ thống quạt tăng áp,... tùy theo quyền hạn được phân cấp.

1.1.3. Giảm thiểu sai sót do con người

Trong thực tế, hầu hết các hệ thống phòng cháy chữa cháy (PCCC) tại các tòa nhà, nhà xưởng hay khu dân cư thường được sử dụng cho lực lượng bảo vệ, nhân viên kỹ thuật tòa nhà hoặc các nhân viên không chuyên sâu về công nghệ. Trong những tình huống khẩn cấp như cháy nổ, tâm lý hoảng loạn có thể khiến người vận hành dễ xảy ra sai sót nếu giao diện hệ thống được thiết kế phức tạp

hoặc khó sử dụng. Vì vậy, việc thiết kế giao diện người dùng (UI) khoa học và trực quan có vai trò quan trọng trong việc hỗ trợ người vận hành thao tác chính xác và nhanh chóng.

UI được thiết kế hợp lý giúp hạn chế nhầm lẫn trong các thao tác nguy hiểm. Các chức năng quan trọng như tắt bơm chữa cháy, tắt toàn bộ hệ thống hoặc reset hệ thống cần được tách biệt rõ ràng so với các thao tác thông thường. Đối với những lệnh có ảnh hưởng lớn đến hoạt động của hệ thống, giao diện nên yêu cầu xác nhận nhiều bước hoặc chỉ cho phép thực hiện bởi người dùng có quyền hạn cao hơn nhằm tránh các thao tác nhầm lẫn.

Giao diện còn có thể hướng dẫn trực tiếp cho người vận hành ngay trên màn hình. Khi có tín hiệu báo cháy, hệ thống có thể hiển thị quy trình xử lý cơ bản, bao gồm các bước như cần thông báo cho bộ phận nào, cần kiểm tra khu vực nào trước, và các thao tác cần thực hiện theo thứ tự. Việc sử dụng các biểu tượng trực quan, hình ảnh minh họa hoặc hướng dẫn ngắn gọn cũng giúp hỗ trợ những người vận hành ít kinh nghiệm có thể nhanh chóng hiểu và thực hiện đúng quy trình.

UI hiện đại còn giúp giảm yêu cầu người vận hành phải ghi nhớ các mã lỗi hoặc ký hiệu kỹ thuật phức tạp. Thay vì chỉ hiển thị các mã lỗi ngắn gọn như “Error 03”, hệ thống có thể diễn giải cụ thể thành thông báo dễ hiểu, ví dụ: “Mất kết nối đầu báo khu vực X – cần kiểm tra tủ tầng Y”. Nhờ đó, người vận hành có thể nhanh chóng nhận biết nguyên nhân sự cố và thực hiện kiểm tra hoặc xử lý phù hợp. Một minh chứng thực tế là hệ thống báo cháy tự động của Siemens tích hợp phần mềm giám sát FMS8000, với giao diện đồ họa trực quan cho phép hiển thị sơ đồ tòa nhà dạng vector, tình trạng thiết bị và lịch sử theo thời gian thực. Khi có tín hiệu báo cháy, hệ thống không chỉ làm nổi bật vị trí khu vực bị ảnh hưởng mà còn cung cấp các bước thao tác xử lý gợi ý giúp người vận hành thực hiện đúng trình tự mà không cần ghi nhớ sâu các chi tiết kỹ thuật phức tạp và khó hiểu, qua đó giảm thiểu đáng kể sai sót do con người trong tình huống khẩn cấp.

Nhờ những đặc điểm trên, việc nghiên cứu và thiết kế giao diện người dùng cho ứng dụng giám sát, phân tích dữ liệu và cảnh báo thông minh không chỉ giúp nâng cao tính thân thiện và dễ sử dụng của hệ thống PCCC mà còn góp phần giảm thiểu rủi ro do yếu tố con người, đảm bảo hệ thống hoạt động đúng theo thiết kế trong các tình huống gặp sự cố khẩn cấp.

1.1.4. Nâng cao hiệu quả quản lý, bảo trì và phân tích dữ liệu

Đối với các hệ thống phòng cháy chữa cháy (PCCC) hiện đại, ứng dụng giám sát, phân tích dữ liệu và cảnh báo thông minh không chỉ đóng vai trò hiển thị thông tin và hỗ trợ thao tác vận hành mà còn là công cụ quan trọng phục vụ công tác giám sát, bảo trì và quản lý rủi ro. Thông qua giao diện trực quan, người quản lý có thể theo dõi tình trạng hoạt động của toàn bộ hệ thống và đưa ra các quyết định kịp thời nhằm đảm bảo hệ thống luôn hoạt động ổn định, phù hợp với mục tiêu của đề tài nghiên cứu.

Giao diện người dùng hỗ trợ giám sát tình trạng thiết bị theo thời gian thực. Hệ thống có thể hiển thị trạng thái kết nối của các thiết bị như đầu báo cháy, van điều khiển, bơm chữa cháy và các tủ điều khiển phụ. Ứng dụng có thể hiển thị trạng thái của cảm biến, tuổi thọ thiết bị hay lịch sử lỗi, giúp người dùng dễ dàng phát hiện các thiết bị có dấu hiệu bất thường để tiến hành kiểm tra hoặc thay thế. Giao diện người dùng còn hỗ trợ bảo trì chủ động. Hệ thống có thể tổng hợp và thống kê tần suất báo lỗi theo từng khu vực hoặc từng thiết bị, từ đó đưa ra gợi ý về những vị trí cần được kiểm tra hoặc bảo trì sớm. Nhờ đó, các vấn đề tiềm ẩn có thể được phát hiện và xử lý trước khi gây ra báo động giả hoặc làm ảnh hưởng đến khả năng hoạt động của hệ thống khi xảy ra sự cố cháy thực sự.

Giao diện ứng dụng giám sát và phân tích dữ liệu còn hỗ trợ phân tích dữ liệu phục vụ quản lý rủi ro. Các dữ liệu thu thập từ hệ thống có thể được biểu diễn dưới dạng biểu đồ hoặc bảng thống kê, ví dụ như số lần báo cháy giả theo tháng, theo khu vực hoặc theo từng đơn vị sử dụng mặt bằng. Dựa trên các dữ liệu này, ban quản lý tòa nhà có thể đánh giá những khu vực có nguy cơ cháy cao để tăng cường công tác tuyên truyền, kiểm tra an toàn hoặc cải thiện các giải pháp phòng cháy chữa cháy phù hợp.

Như vậy, trong phạm vi đề tài “*Nghiên cứu, thiết kế ứng dụng giám sát, phân tích dữ liệu và cảnh báo thông minh*”, giao diện người dùng không chỉ là phương tiện hiển thị thông tin mà còn đóng vai trò là công cụ hỗ trợ quản lý và ra quyết định. Tuy nhiên, các chức năng giám sát và phân tích dữ liệu này chỉ thực sự phát huy hiệu quả khi được trình bày thông qua một giao diện trực quan, dễ hiểu và thuận tiện cho người sử dụng. Ngoài ra, việc ứng dụng các công nghệ phân tích dữ liệu còn cho phép hệ thống hỗ trợ bảo trì dự đoán. Dựa trên lịch sử hoạt động và các thông số thu thập được từ cảm biến, hệ thống có thể đưa ra cảnh báo sớm đối với các thiết bị có dấu hiệu hoạt động bất thường hoặc

nguy cơ hư hỏng cao. Nhờ đó, công tác bảo trì không chỉ dừng lại ở hình thức kiểm tra định kỳ mà còn trở nên chủ động và hiệu quả hơn, góp phần giảm thiểu chi phí vận hành và hạn chế nguy cơ hệ thống gặp lỗi trong tình huống khẩn cấp.

1.1.5. Kết luận

Giao diện người dùng đóng vai trò như một “cầu nối” giữa con người và hệ thống phòng cháy chữa cháy. Thông qua giao diện, người vận hành có thể theo dõi trạng thái hệ thống, tiếp nhận cảnh báo và thực hiện các thao tác xử lý khi xảy ra sự cố. Một giao diện được thiết kế khoa học và trực quan giúp người dùng nhận biết sự cố nhanh chóng, giảm thiểu sai sót trong thao tác, đồng thời hỗ trợ tối ưu hóa công tác giám sát và phân tích dữ liệu.

Một giao diện được thiết kế khoa học và trực quan mang lại nhiều lợi ích quan trọng. Trước hết, UI giúp người dùng nhận biết sự cố nhanh chóng thông qua các thông tin hiển thị rõ ràng và các tín hiệu cảnh báo trực quan. Đồng thời, giao diện hợp lý còn giúp giảm thiểu các sai sót trong thao tác, đặc biệt trong những tình huống khẩn cấp khi người vận hành có thể bị áp lực hoặc căng thẳng.

UI còn hỗ trợ tối ưu hóa công tác giám sát, bảo trì và phân tích dữ liệu của hệ thống. Nhờ việc hiển thị thông tin theo thời gian thực và cung cấp các công cụ thống kê, phân tích, người quản lý có thể theo dõi tình trạng hoạt động của thiết bị, phát hiện sớm các nguy cơ tiềm ẩn và đưa ra các biện pháp xử lý kịp thời. Điều này góp phần nâng cao hiệu quả vận hành của hệ thống và giảm thiểu thiệt hại về người và tài sản khi xảy ra cháy nổ.

Vì vậy, trong quá trình nghiên cứu và thiết kế ứng dụng giám sát, phân tích dữ liệu và cảnh báo thông minh cho hệ thống PCCC, việc đầu tư thiết kế giao diện người dùng một cách bài bản là yêu cầu cần thiết. UI không chỉ là yếu tố liên quan đến tính thẩm mỹ của hệ thống mà còn là thành phần quan trọng ảnh hưởng trực tiếp đến hiệu quả vận hành và khả năng phản ứng trong các tình huống khẩn cấp.

1.2. XU HƯỚNG ỨNG DỤNG (WEB/APP) TRONG GIÁM SÁT TỪ XA

Trong bối cảnh công nghệ thông tin và Internet ngày càng phát triển, việc ứng dụng các nền tảng Web và ứng dụng di động trong giám sát hệ thống phòng cháy chữa cháy (PCCC) đang trở thành một xu hướng tất yếu.

1.2.1. Nhu cầu giám sát từ xa trong PCCC

Trong bối cảnh các công trình hiện đại ngày càng có quy mô lớn và phức tạp, nhu cầu giám sát hệ thống PCCC một cách hiệu quả và liên tục ngày càng trở nên cấp thiết. Nhiều tòa nhà, khu công nghiệp hoặc cụm nhà máy được xây dựng với nhiều khu vực và nhiều tòa nhà riêng biệt, khiến việc theo dõi toàn bộ hệ thống chỉ thông qua một phòng điều khiển trung tâm gặp nhiều khó khăn. Trong trường hợp xảy ra sự cố, việc xác định nhanh vị trí và tình trạng của hệ thống có thể bị hạn chế nếu không có công cụ giám sát phù hợp.

Các đơn vị quản lý vận hành hoặc chủ đầu tư thường sở hữu và quản lý nhiều công trình khác nhau. Do đó, việc theo dõi trạng thái hoạt động của hệ thống PCCC tại từng địa điểm một cách riêng lẻ sẽ tốn nhiều thời gian và nguồn lực. Vì vậy, nhu cầu xây dựng một hệ thống giám sát tập trung, cho phép theo dõi đồng thời nhiều công trình trên cùng một nền tảng là rất cần thiết.

Đối với lãnh đạo và cán bộ kỹ thuật, việc nắm bắt tình trạng an toàn PCCC của công trình mọi lúc, mọi nơi cũng là một yêu cầu quan trọng. Trong nhiều trường hợp, họ không trực tiếp có mặt tại hiện trường nhưng vẫn cần nhận được thông tin cảnh báo kịp thời để đưa ra chỉ đạo hoặc phương án xử lý phù hợp.

Một yếu tố khác cần được xem xét là nhu cầu tối ưu chi phí nhân sự vận hành hệ thống. Thay vì bố trí nhiều nhân viên trực tại từng công trình riêng lẻ, các đơn vị quản lý có thể triển khai mô hình trung tâm giám sát tập trung, từ đó giảm bớt chi phí nhân lực mà vẫn đảm bảo khả năng theo dõi và phản ứng nhanh khi có sự cố.

Chính những nhu cầu thực tế trên đã thúc đẩy sự phát triển của các giải pháp giám sát hệ thống PCCC từ xa theo thời gian thực. Các hệ thống này thường được xây dựng trên nền tảng Web và ứng dụng di động (Mobile), cho phép người dùng theo dõi trạng thái thiết bị, nhận cảnh báo sự cố và quản lý hệ thống một cách thuận tiện và linh hoạt.

1.2.2. Ứng dụng Web trong giám sát hệ thống PCCC

Ứng dụng Web trong hệ thống giám sát phòng cháy chữa cháy (PCCC) hiện đại đóng vai trò như bảng điều khiển trung tâm dành cho cán bộ kỹ thuật và quản lý vận hành. Qua giao diện này, người dùng theo dõi toàn bộ trạng thái hệ thống, nhận cảnh báo và quản lý thiết bị từ một nền tảng tập trung.

Ưu điểm là truy cập dễ dàng qua trình duyệt, không cần cài đặt phần mềm phức tạp. Điều này làm việc triển khai và sử dụng trở nên thuận tiện, đặc biệt khi quản lý nhiều công trình.

Giao diện Web lý tưởng cho màn hình lớn tại phòng điều khiển, hiển thị đồng thời bản đồ, sơ đồ tòa nhà, trạng thái thiết bị và biểu đồ dữ liệu. Nó dễ tích hợp với BMS, SCADA, hệ thống an ninh, camera.

Chức năng của ứng dụng cung cấp tổng quan trạng thái toàn hệ thống trên một màn hình. Khi sự cố xảy ra, người dùng xem chi tiết như thời gian, vị trí cảm biến, loại cảnh báo - thậm chí kèm hình ảnh và video từ camera ở hệ thống nâng cao.

Ứng dụng còn hỗ trợ thống kê báo cáo: lịch sử báo cháy, lỗi hệ thống, bảo trì thiết bị. Nó quản lý người dùng, phân quyền theo vai trò để đảm bảo bảo mật. Xu hướng hiện nay dùng công nghệ SPA, WebSocket hoặc MQTT cho cập nhật thời gian thực, mang lại trải nghiệm mượt mà và phản hồi nhanh.

Ngoài chức năng giám sát, ứng dụng Web còn hỗ trợ quản lý nhật ký vận hành và lịch sử thao tác của người dùng. Mọi hoạt động như đăng nhập hệ thống, xác nhận cảnh báo hoặc thay đổi trạng thái thiết bị đều được lưu lại nhằm phục vụ công tác kiểm tra và đảm bảo tính minh bạch trong quá trình vận hành.

Trong các hệ thống quy mô lớn, giao diện Web còn hỗ trợ phân quyền người dùng theo nhiều cấp độ khác nhau như quản trị viên, kỹ thuật viên hoặc nhân viên giám sát. Việc phân quyền này giúp đảm bảo tính bảo mật của hệ thống, hạn chế các thao tác không phù hợp có thể ảnh hưởng đến hoạt động của hệ thống PCCC.

1.2.3. Mobile App trong cảnh báo và hỗ trợ xử lý

Bên cạnh ứng dụng Web, ứng dụng di động đóng vai trò quan trọng trong việc truyền tải thông tin cảnh báo đến người dùng một cách nhanh chóng và kịp thời. Nhờ tính linh hoạt của thiết bị di động, người dùng có thể nhận thông tin về tình trạng hệ thống phòng cháy chữa cháy mọi lúc, mọi nơi.

Một trong những chức năng quan trọng của ứng dụng Mobile là cảnh báo tức thời. Khi hệ thống phát hiện dấu hiệu cháy, rò rỉ khí gas hoặc các sự cố kỹ thuật khác, ứng dụng sẽ gửi thông báo đẩy (push notification) đến điện thoại của người dùng. Điều này giúp đảm bảo các thông tin quan trọng được truyền tải nhanh chóng và giảm nguy cơ bỏ sót cảnh báo. Ngoài ra, ứng dụng còn cho

phép người dùng xác nhận đã nhận cảnh báo, giúp hệ thống theo dõi tình trạng phản hồi của người vận hành.

Ứng dụng Mobile cũng hỗ trợ hiệu quả trong quá trình xử lý sự cố tại hiện trường. Người dùng có thể truy cập các thông tin cần thiết như sơ đồ thoát nạn, vị trí các thiết bị PCCC gần nhất (bình chữa cháy, họng nước chữa cháy...), từ đó hỗ trợ việc xử lý tình huống khẩn cấp nhanh chóng hơn. Bên cạnh đó, ứng dụng còn cho phép người dùng chụp ảnh, ghi chú và cập nhật tình trạng xử lý sự cố, giúp đồng bộ thông tin về trung tâm giám sát.

Ứng dụng Mobile mang lại sự tiện lợi cho nhiều nhóm đối tượng sử dụng khác nhau. Đối với kỹ thuật viên, ứng dụng giúp tiếp nhận các nhiệm vụ kiểm tra, bảo trì hoặc sửa chữa thiết bị. Đối với ban quản lý tòa nhà, ứng dụng hỗ trợ theo dõi tình trạng sự cố và tiến độ khắc phục theo thời gian thực. Trong khi đó, lực lượng PCCC cơ sở có thể sử dụng ứng dụng như một công cụ hỗ trợ nhanh trong các tình huống khẩn cấp.

Hiện nay, xu hướng phát triển các ứng dụng Mobile trong lĩnh vực giám sát PCCC là xây dựng ứng dụng đa nền tảng hoạt động trên cả Android và iOS. Một số hệ thống còn hỗ trợ chế độ làm việc ngoại tuyến, cho phép người dùng ghi nhận dữ liệu ngay cả khi mất kết nối mạng và tự động đồng bộ lại khi có Internet.

Bên cạnh chức năng nhận cảnh báo tức thời, ứng dụng Mobile còn giúp nâng cao khả năng phối hợp giữa các bộ phận khi xảy ra sự cố. Thông qua kết nối Internet, các thông tin liên quan đến đám cháy hoặc sự cố kỹ thuật có thể được chia sẻ nhanh chóng giữa nhân viên kỹ thuật, lực lượng bảo vệ và ban quản lý tòa nhà. Điều này giúp các bộ phận liên quan nắm bắt tình hình đồng bộ và triển khai phương án xử lý hiệu quả hơn.

Ứng dụng Mobile hiện đại còn hỗ trợ định vị vị trí người dùng và hiển thị tuyến đường tiếp cận khu vực sự cố trong các công trình lớn. Tính năng này đặc biệt hữu ích đối với lực lượng cứu hộ hoặc kỹ thuật viên khi cần di chuyển nhanh đến vị trí xảy ra cháy trong thời gian ngắn nhất.

Ngoài ra, nhiều ứng dụng còn tích hợp chức năng kiểm tra định kỳ thiết bị PCCC ngay trên điện thoại di động. Kỹ thuật viên có thể sử dụng ứng dụng để quét mã QR của thiết bị, cập nhật tình trạng hoạt động hoặc ghi nhận kết quả kiểm tra trực tiếp tại hiện trường. Dữ liệu sau đó sẽ được đồng bộ về hệ thống

quản lý trung tâm, giúp giảm thiểu việc ghi chép thủ công và nâng cao hiệu quả quản lý thiết bị.

Nhờ tính linh hoạt và khả năng hoạt động trên thiết bị di động, Mobile App đang trở thành một thành phần quan trọng trong các hệ thống giám sát PCCC hiện đại, góp phần nâng cao khả năng phản ứng nhanh và tối ưu hóa công tác vận hành trong thực tế.

1.2.4. Các xu hướng công nghệ nổi bật

Trong bối cảnh phát triển của các mô hình ngôi nhà thông minh việc giám sát hệ thống phòng cháy chữa cháy từ xa đang ngày càng gắn liền với các công nghệ hiện đại. Việc ứng dụng các công nghệ này không chỉ giúp nâng cao hiệu quả giám sát mà còn hỗ trợ phân tích dữ liệu và dự đoán các nguy cơ tiềm ẩn, từ đó góp phần đảm bảo an toàn cho con người và tài sản.

Internet of Things (IoT) đóng vai trò nền tảng trong việc xây dựng các hệ thống giám sát PCCC thông minh. Các thiết bị cảm biến như cảm biến khói, cảm biến nhiệt độ, cảm biến khí gas hoặc các thiết bị điều khiển PCCC được kết nối mạng và có khả năng truyền dữ liệu liên tục về máy chủ hoặc hệ thống quản lý. Nhờ đó, các thông tin về trạng thái thiết bị và môi trường có thể được cập nhật theo thời gian thực.



Hình 1.1. Công nghệ IoT

Điện toán đám mây cho phép lưu trữ và xử lý dữ liệu tập trung trên các máy chủ từ xa. Việc sử dụng nền tảng đám mây giúp hệ thống dễ dàng mở rộng

quy mô khi số lượng thiết bị hoặc công trình cần giám sát tăng lên, đồng thời giúp việc truy cập và quản lý dữ liệu trở nên thuận tiện hơn.

Sự phát triển của phân tích dữ liệu lớn và trí tuệ nhân tạo (AI) cũng mở ra nhiều khả năng mới cho các hệ thống PCCC. Thông qua việc phân tích dữ liệu lịch sử và các mẫu hoạt động của hệ thống, các thuật toán có thể phát hiện các dấu hiệu bất thường, đưa ra cảnh báo sớm hoặc gợi ý các kế hoạch bảo trì dự đoán nhằm giảm thiểu nguy cơ xảy ra sự cố.

Một xu hướng quan trọng khác là tích hợp đa hệ thống trên cùng một nền tảng quản lý. Hệ thống PCCC có thể được kết nối và tích hợp với các hệ thống khác như hệ thống an ninh, giám sát môi trường, hệ thống quản lý tòa nhà hoặc hệ thống điều khiển tự động. Việc tích hợp này giúp tạo ra một hệ thống quản lý tổng thể, cho phép giám sát và điều khiển nhiều chức năng khác nhau trong cùng một giao diện.

Nhờ sự kết hợp của các công nghệ trên, các hệ thống giám sát PCCC hiện đại không chỉ dừng lại ở việc phát hiện và cảnh báo cháy mà còn hướng tới một hệ thống quản lý an toàn thông minh, tự động và có khả năng dự đoán rủi ro trong tương lai.

1.2.5. Lợi ích và thách thức

Lợi ích:

Hệ thống giám sát PCCC từ xa giúp nâng cao khả năng giám sát liên tục 24/7 mà không cần bố trí quá nhiều nhân lực trực tại từng công trình. Nhờ việc kết nối các thiết bị cảm biến và hệ thống cảnh báo với nền tảng quản lý tập trung, người vận hành có thể theo dõi tình trạng hệ thống mọi lúc thông qua máy tính hoặc thiết bị di động.

Hệ thống còn giúp tăng tốc độ phản ứng khi xảy ra sự cố. Khi phát hiện dấu hiệu cháy hoặc rò rỉ khí gas, thông tin cảnh báo sẽ được gửi ngay lập tức đến đúng người phụ trách thông qua các kênh như ứng dụng di động, tin nhắn hoặc email. Điều này giúp rút ngắn thời gian phát hiện và xử lý sự cố, từ đó giảm thiểu thiệt hại về người và tài sản.

Việc lưu trữ và phân tích dữ liệu từ hệ thống cũng giúp tối ưu hóa công tác vận hành và bảo trì thiết bị. Dựa trên các dữ liệu thực tế như tần suất lỗi, lịch sử cảnh báo hoặc trạng thái hoạt động của thiết bị, người quản lý có thể đưa ra các kế hoạch bảo trì phù hợp, nâng cao độ tin cậy và tuổi thọ của hệ thống.

Thách thức:

Bên cạnh những lợi ích trên, việc triển khai hệ thống giám sát PCCC từ xa cũng đối mặt với một số thách thức nhất định. Trước hết là yêu cầu cao về bảo mật và an toàn thông tin. Do hệ thống được kết nối với mạng Internet, cần có các cơ chế bảo vệ nhằm tránh các nguy cơ bị truy cập trái phép hoặc can thiệp từ xa vào hệ thống.

Một thách thức khác là sự phụ thuộc vào hạ tầng kỹ thuật, bao gồm mạng Internet, nguồn điện và hệ thống máy chủ. Nếu các thành phần này gặp sự cố, khả năng truyền dữ liệu và giám sát từ xa có thể bị ảnh hưởng.

Việc chuyển đổi từ hệ thống giám sát truyền thống sang hệ thống giám sát thông minh cũng đòi hỏi quá trình đào tạo người sử dụng. Người vận hành cần được hướng dẫn và làm quen với các giao diện quản lý mới, các phương thức nhận cảnh báo cũng như các quy trình xử lý sự cố trong môi trường công nghệ hiện đại. Mặc dù còn tồn tại một số thách thức, nhưng những lợi ích mà hệ thống giám sát PCCC từ xa mang lại là rất lớn, góp phần nâng cao hiệu quả quản lý và đảm bảo an toàn cho các công trình trong bối cảnh phát triển của các hệ thống thông minh hiện nay.

Như vậy, Chương 1 đã trình bày tổng quan về ứng dụng giám sát và cảnh báo thông minh trong hệ thống PCCC, trong đó nhấn mạnh vai trò quan trọng của giao diện người dùng trong việc nâng cao hiệu quả vận hành và giảm thiểu sai sót. Đồng thời, chương cũng phân tích xu hướng ứng dụng các nền tảng Web và Mobile trong giám sát từ xa, giúp tăng khả năng phản ứng nhanh khi xảy ra sự cố. Bên cạnh đó, các công nghệ hiện đại như IoT, Cloud và AI đã được đề cập như những nền tảng cốt lõi cho việc xây dựng hệ thống PCCC thông minh. Qua đó, chương làm rõ lợi ích, thách thức và định hướng phát triển của các hệ thống giám sát trong thực tế.

CHƯƠNG 2. CÔNG NGHỆ PHÁT TRIỂN ỨNG DỤNG VÀ XỬ LÝ DỮ LIỆU

Sau khi trình bày tổng quan về vai trò của giao diện người dùng và xu hướng giám sát từ xa trong hệ thống PCCC ở chương 1 thì nội dung chương 2 tập trung trình bày các công nghệ cốt lõi trong xây dựng hệ thống giám sát và cảnh báo PCCC, bao gồm framework phát triển Web/Mobile, thư viện trực quan hóa dữ liệu và các thuật toán xử lý dữ liệu. Nội dung chương làm rõ cách thiết kế giao diện, hiển thị dữ liệu trực quan và xây dựng cơ chế cảnh báo thông minh, tạo nền tảng cho hệ thống hoạt động hiệu quả, thời gian thực và hỗ trợ ra quyết định chính xác.

2.1. CÁC FRAMEWORK PHÁT TRIỂN WEB/MOBILE

2.1.1. Framework phát triển Web

Từ “Framework” có nguồn gốc từ “bộ khung”. Trong lĩnh vực lập trình, Framework là một tập hợp các thư viện và công cụ đã được xây dựng sẵn, cung cấp các chức năng và cấu trúc cơ bản để giúp lập trình viên phát triển phần mềm hoặc ứng dụng web một cách nhanh chóng và hiệu quả. Nó giống như một bộ công cụ chuyên dụng cho từng mục đích, giúp giảm thiểu việc phải viết lại mã nguồn từ đầu và tăng tính nhất quán trong quá trình phát triển.



Hình 2.1. Framework

Ví dụ, để thiết kế một website thương mại điện tử, nếu không có framework, lập trình viên phải tự tạo ra các thành phần như giỏ hàng, hệ thống thanh toán, trang sản phẩm, và quản lý người dùng, sau đó tích hợp chúng lại với nhau. Tuy nhiên, với framework sẵn có lập trình viên chỉ cần viết lệnh để sử dụng các thành phần này từ framework và kết hợp chúng lại. Framework giống như các vật liệu xây dựng như gạch, cát, xi măng, và sắt, mà tùy theo cách thiết kế của kiến trúc sư có thể tạo ra các ngôi nhà với hình dạng khác nhau. Theo cách này, dù không có kỹ năng lập trình cao, người lập trình vẫn có thể tạo ra sản phẩm chất lượng bằng cách sử dụng framework và tuân thủ các quy tắc của nó.



Hình 2.2. Một số Web Framework phổ biến

Web framework là một khung phần mềm cung cấp các công cụ và thư viện giúp lập trình viên xây dựng và phát triển các ứng dụng web một cách hiệu quả. Nó bao gồm các công cụ để quản lý và xử lý yêu cầu HTTP, quản lý session và cookie, kết nối cơ sở dữ liệu, và xây dựng giao diện người dùng. Bên cạnh đó, Web framework còn cung cấp các quy ước và mẫu thiết kế giúp định hướng cách tổ chức và triển khai mã nguồn một cách có tổ chức. Nhờ framework, lập trình viên có thể tập trung vào logic của ứng dụng mà không cần phải lo lắng quá nhiều về các chi tiết kỹ thuật cơ bản.

Một số framework phổ biến hiện nay gồm:

- React: Là một thư viện JavaScript mạnh mẽ dùng để xây dựng giao diện người dùng. React cho phép cập nhật dữ liệu theo thời gian thực mà không cần tải lại trang, rất phù hợp với các hệ thống giám sát như IoT.
- Vue.js: Là một framework nhẹ, dễ học, phù hợp với các dự án vừa và nhỏ. Vue.js hỗ trợ xây dựng giao diện nhanh chóng với cú pháp đơn giản.
- Node.js: Là môi trường chạy JavaScript phía server, thường được sử dụng để xây dựng backend. Node.js có khả năng xử lý dữ liệu nhanh, hỗ trợ tốt các giao thức thời gian thực như MQTT.



Hình 2.3. Các Framework phát triển Web

Trong hệ thống cảnh báo cháy, nền tảng Web có thể đảm nhận các chức năng như:

- Hiện thị dữ liệu cảm biến (nhiệt độ, khí gas, khói, lửa) theo thời gian thực.
- Giám sát hệ thống trên máy tính hoặc laptop.
- Quản lý dữ liệu tổng thể.
- Hỗ trợ mở rộng hệ thống trong tương lai.
- Web đóng vai trò là cầu nối giữa hệ thống IoT và người dùng, giúp tổng hợp được các dữ liệu từ nhiều thiết bị và hiển thị một cách trực quan nhất.

2.1.2. Framework phát triển ứng dụng Mobile

Trong phạm vi đề tài, ứng dụng Mobile được xây dựng theo hướng phát triển Native Android, sử dụng môi trường Android Studio và ngôn ngữ lập trình Kotlin. Việc lựa chọn phương pháp phát triển Native không chỉ đảm bảo hiệu

năng cao mà còn giúp ứng dụng khai thác tối đa các tài nguyên phần cứng của thiết bị di động, từ đó đáp ứng tốt yêu cầu về tốc độ và độ ổn định của hệ thống cảnh báo cháy.

Phát triển ứng dụng Mobile Native là phương pháp xây dựng ứng dụng dành riêng cho từng hệ điều hành cụ thể như Android hoặc iOS, sử dụng các công cụ và ngôn ngữ chính thức do nhà phát triển nền tảng cung cấp. So với các phương pháp phát triển đa nền tảng, cách tiếp cận này mang lại hiệu suất xử lý cao hơn, khả năng truy cập trực tiếp vào phần cứng tốt hơn và độ ổn định cao hơn. Đối với các hệ thống yêu cầu phản hồi nhanh như hệ thống giám sát và cảnh báo cháy, việc lựa chọn phát triển Native Android là một giải pháp phù hợp và hiệu quả.



Hình 2.4. Giao diện môi trường phát triển Android Studio

Android Studio là môi trường phát triển tích hợp chính thức dành cho hệ điều hành Android, được phát triển dựa trên nền tảng IntelliJ IDEA. Công cụ này cung cấp đầy đủ các chức năng hỗ trợ lập trình hiện đại như trình soạn thảo mã nguồn thông minh với khả năng gợi ý tự động, giúp giảm thiểu lỗi trong quá trình viết code. Bên cạnh đó, Android Studio tích hợp trình giả lập Android Emulator, cho phép kiểm thử ứng dụng trên nhiều loại thiết bị khác nhau mà không cần sử dụng thiết bị thật. Ngoài ra, hệ thống debug mạnh mẽ giúp phát hiện và xử lý lỗi nhanh chóng, trong khi công cụ Layout Editor hỗ trợ thiết kế giao diện trực quan thông qua thao tác kéo thả. Không những vậy, Android Studio còn hỗ trợ tích hợp các dịch vụ như Firebase, MQTT giúp đơn giản hóa việc xây dựng các ứng dụng IoT có yêu cầu về đồng bộ dữ liệu và thông báo theo thời gian thực.

Kotlin là ngôn ngữ lập trình hiện đại, được Google công nhận là ngôn ngữ chính thức cho phát triển ứng dụng Android. So với các ngôn ngữ truyền thống như Java, Kotlin có cú pháp ngắn gọn và rõ ràng hơn, giúp nâng cao khả năng đọc hiểu và bảo trì mã nguồn. Một trong những ưu điểm nổi bật của Kotlin là cơ chế Null Safety, giúp hạn chế tối đa các lỗi liên quan đến con trỏ null – một trong những nguyên nhân phổ biến gây lỗi ứng dụng. Bên cạnh đó, Kotlin hỗ trợ lập trình bất đồng bộ thông qua Coroutines, cho phép xử lý hiệu quả các tác vụ như nhận dữ liệu từ cảm biến hoặc giao tiếp với máy chủ mà không làm ảnh hưởng đến hiệu năng chung của ứng dụng. Ngoài ra, Kotlin hoàn toàn tương thích với Java, giúp dễ dàng tận dụng các thư viện và tài nguyên đã có sẵn.



Hình 2.5. Kotlin - Ngôn ngữ lập trình hiện đại sử dụng trong ứng dụng Mobile Android

Về mặt kiến trúc, ứng dụng Mobile trong hệ thống được thiết kế theo mô hình MVVM (Model – View – ViewModel), một trong những kiến trúc phổ biến và hiệu quả trong phát triển Android hiện đại. Trong đó, Model chịu trách nhiệm quản lý dữ liệu và tương tác với hệ thống IoT hoặc máy chủ; View đảm nhiệm việc hiển thị giao diện và tương tác trực tiếp với người dùng; còn ViewModel đóng vai trò trung gian, xử lý logic nghiệp vụ và cung cấp dữ liệu cho View. Việc áp dụng mô hình MVVM giúp tách biệt rõ ràng giữa phần giao diện và logic xử lý, từ đó nâng cao khả năng bảo trì, mở rộng hệ thống và hỗ trợ kiểm thử một cách hiệu quả.

Trong tổng thể hệ thống, ứng dụng Mobile đóng vai trò là thành phần quan

trọng trong việc kết nối người dùng với hệ thống IoT. Ứng dụng có khả năng tiếp nhận và hiển thị dữ liệu từ các cảm biến theo thời gian thực, giúp người dùng dễ dàng theo dõi trạng thái hoạt động của hệ thống. Đồng thời, khi phát hiện các dấu hiệu bất thường như nhiệt độ tăng cao, nồng độ khí gas, khói vượt ngưỡng hoặc xuất hiện ngọn lửa, hệ thống sẽ gửi cảnh báo ngay lập tức đến người dùng thông qua ứng dụng. Ngoài ra, ứng dụng còn cho phép người dùng thực hiện các thao tác điều khiển thiết bị từ xa, góp phần nâng cao tính linh hoạt và hiệu quả trong quá trình vận hành hệ thống. Đặc biệt, tính năng push notification cho phép gửi thông báo ngay cả khi ứng dụng không đang hoạt động, đảm bảo người dùng luôn nhận được cảnh báo kịp thời trong các tình huống khẩn cấp.

2.1.3. So sánh và lựa chọn công nghệ phù hợp

Trong quá trình xây dựng hệ thống giám sát và cảnh báo cháy, rò rỉ gas dựa trên công nghệ IoT, việc lựa chọn công nghệ phát triển phù hợp là yếu tố vô cùng quan trọng nhằm đảm bảo hiệu quả, hiệu năng, độ ổn định cũng như khả năng mở rộng của hệ thống. Dựa trên các nền tảng đã phân tích, có thể so sánh giữa hai hướng phát triển chính là nền tảng Web và ứng dụng Mobile, đồng thời đánh giá giữa phương pháp phát triển Native Android và đa nền tảng (Cross-platform).

Bảng 2.1. So sánh nền tảng Web và nền tảng Mobile

Tiêu chí	Nền tảng Web	Nền tảng Mobile
Khả năng truy cập	Truy cập qua trình duyệt, không cần cài đặt	Truy cập qua ứng dụng, cần cài đặt ứng dụng
Thiết bị sử dụng	Máy tính, laptop	Điện thoại thông minh
Hiệu năng	Phụ thuộc trình duyệt	Cao, tối ưu tốt cho thiết bị
Cảnh báo thời gian thực	Hạn chế do phụ thuộc vào trình duyệt	Rất tốt nhờ sự hỗ trợ push notification tức thời
Trải nghiệm người dùng	Trung bình	Cao, tương tác trực tiếp
Khả năng mở rộng	Tốt, dễ nâng cấp và triển khai trên nhiều hệ thống	Tốt, có thể mở rộng tính năng và tích hợp dịch vụ

Ứng dụng trong hệ thống	Giám sát tổng thể, quản lý dữ liệu tập trung	Cảnh báo nhanh, điều khiển thiết bị từ xa
-------------------------	--	---

Bảng 2.2. So sánh giữa Native Android (Kotlin) và Cross-platform

Tiêu chí	Native Android (Kotlin)	Cross-platform
Hiệu năng	Cao, tối ưu hóa tốt cho hệ điều hành Android	Trung bình, phụ thuộc vào lớp trung gian
Truy cập phần cứng	Trực tiếp, khai thác đầy đủ tài nguyên thiết bị	Hạn chế, phụ thuộc vào framework
Độ ổn định	Cao, ít phụ thuộc bên thứ 3	Phụ thuộc vào framework và thư viện sử dụng
Tốc độ phát triển	Chậm hơn do phát triển riêng từng nền tảng	Nhanh hơn do sử dụng chung một mã nguồn
Chi phí phát triển	Cao hơn do cần đội ngũ chuyên sâu	Thấp hơn nhờ giảm thời gian và nguồn lực phát triển
Khả năng mở rộng	Tốt, dễ tích hợp sâu với hệ thống	Trung bình, bị giới hạn bởi framework
Phù hợp với hệ thống IoT	Rất phù hợp với yêu cầu thời gian thực và độ ổn định cao	Ít phù hợp với hệ thống yêu cầu độ chính xác cao

2.2. CÁC THƯ VIỆN TRỰC QUAN HÓA DỮ LIỆU

2.2.1. Tổng quan về trực quan hóa dữ liệu

Trực quan hóa dữ liệu (Data Visualization) là quá trình biểu diễn thông tin phức tạp dưới dạng hình ảnh dễ nhìn, dễ hiểu. Giúp người xem hiểu rõ hơn về dữ liệu, tổng quan hóa, phát hiện xu hướng và thúc đẩy quyết định thông minh.

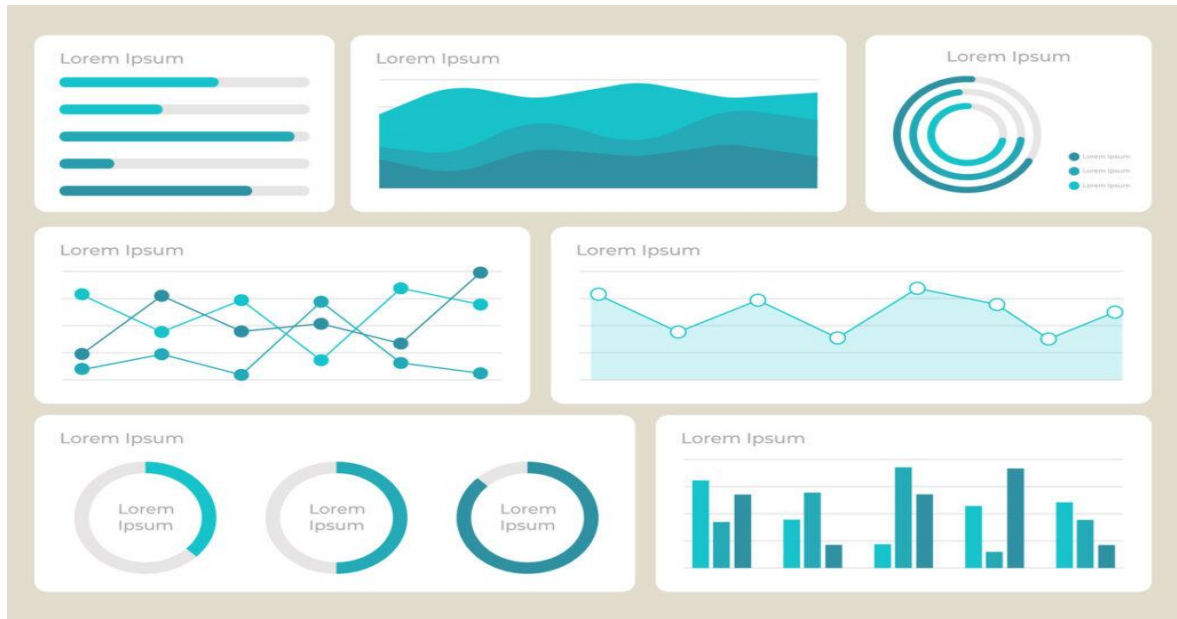
Các khái niệm, thuật ngữ liên quan đến trực quan hóa dữ liệu (được gọi chung là biểu đồ):

- Biểu đồ: Biểu diễn số liệu và thông tin bằng hình ảnh như cột, đường, điểm,

v.v. Thể hiện mối quan hệ, xu hướng, sự khác biệt,...

- Đồ thị: Là một khái niệm toán học biểu diễn một tập hợp các điểm (gọi là đỉnh) được nối với nhau bằng các đoạn thẳng hoặc đoạn cong (gọi là cạnh), thể hiện mối quan hệ, xu hướng,...
- Bản đồ: Hiển thị dữ liệu và thông tin về phân bố địa lý dựa trên vị trí địa lý của các vùng, miền, quốc gia,...
- Biểu đồ: Thể hiện mối quan hệ trong một quá trình, quy trình, mô hình, ...

Ví dụ về Dashboard:



Hình 2.6. Ví dụ về Dashboard

Dashboard là một thuật ngữ trong lĩnh vực trực quan hóa dữ liệu, được hiểu là một giao diện trực quan tập hợp thông tin từ nhiều nguồn dữ liệu khác nhau và hiển thị chúng dưới dạng các biểu đồ, đồ thị, bản đồ và các thành phần trực quan khác. Cung cấp cái nhìn tổng quan về tình hình dữ liệu, giúp người dùng nhanh chóng nhận ra sự khác biệt, xu hướng, mối liên hệ và các thông tin quan trọng khác.

- Tổng quan và dễ nhìn: Bố cục rõ ràng, thông tin sắp xếp theo mức độ, sử dụng màu sắc phù hợp để phân biệt trạng thái (Bình thường, Cảnh báo, Nguy hiểm).
- Tập hợp dữ liệu từ nhiều nguồn: Kết hợp dữ liệu từ cảm biến, cơ sở dữ liệu lịch sử,...
- Trực quan và trực tiếp: Hiển thị trực tiếp thông tin dưới dạng biểu đồ dạng cột giúp người dùng không cần phân tích dữ liệu thô.

- Tùy chỉnh và tương tác: Cho phép người dùng lọc dữ liệu theo thời gian, loại thiết bị hoặc nhấn vào thành phần để xem chi tiết.
- Thời gian thực: Cập nhật liên tục tình trạng hoạt động của hệ thống PCCC, phản ánh kịp thời gian các tín hiệu báo cháy, cảnh báo kỹ thuật, lỗi kết nối,...

Ý nghĩa của trực quan hóa dữ liệu :

- Giúp hiểu rõ hơn về dữ liệu và sự liên hệ giữa các yếu tố.
- Phát hiện thông tin tiềm ẩn và hiển thị mô hình dữ liệu.
- Tối ưu hóa quyết định và dự đoán.

Một số lợi ích của trực quan hóa dữ liệu:

Giúp việc chia sẻ thông tin dễ dàng và nhanh chóng, bỏ qua rào cản, hạn chế về ngôn ngữ, giúp người đọc, người nghe dễ quan sát, dễ tiếp thu.

Giúp chúng ta nắm bắt thông tin được thể hiện dưới các bảng biểu, đồ thị nhanh chóng, đầy đủ về bất kỳ một vấn đề nào, thể hiện trực quan mối liên hệ về những đối tượng nghiên cứu trong dữ liệu, khám phá những sự thật chưa biết, các điểm bất thường và xu hướng biến động của đối tượng nghiên cứu.

Ngày nay các tổ chức đang sử dụng trực quan hóa dữ liệu và các công cụ hỗ trợ để đặt câu hỏi tốt hơn về các vấn đề kinh doanh và đưa ra quyết định tốt hơn. Công nghệ máy tính phát triển và nhiều chương trình, phần mềm hỗ trợ Trực quan hóa dữ liệu ra đời, dễ sử dụng giúp người dùng dễ dàng tìm hiểu thêm về công ty của mình và đưa ra các quyết định kinh doanh dựa trên dữ liệu tốt hơn.

Hỗ trợ kiểm soát, theo dõi các chỉ số về hiệu quả hoạt động, tình hình hoạt động của hệ thống PCCC dựa trên các Dashboard, nhằm cho thấy tầm quan trọng của việc tận dụng, khai thác tài sản dữ liệu để phục vụ ra quyết định.

2.2.2. Các thư viện trực quan hóa dữ liệu

Trong ứng dụng giám sát PCCC, việc trực quan hóa dữ liệu được thực hiện thông qua các thư viện và thành phần giao diện trên nền tảng Android. Dựa vào chương trình đã xây dựng, các thành phần chính gồm:

a. RecyclerView

RecyclerView là thành phần quan trọng được sử dụng để hiển thị danh sách lịch sử cảnh báo trong ứng dụng. Mỗi bản ghi lịch sử bao gồm các thông tin như tên phòng, thời gian, nhiệt độ, giá trị lửa, giá trị gas và trạng thái hoạt động. Nhờ RecyclerView, dữ liệu được trình bày theo dạng danh sách có cấu

trúc rõ ràng, dễ theo dõi và tối ưu khi số lượng bản ghi lớn. Đây là thành phần phù hợp với các hệ thống giám sát cần hiển thị dữ liệu cập nhật liên tục.

b. Adapter trong RecyclerView

Để quản lý và hiển thị dữ liệu trên RecyclerView, hệ thống sử dụng lớp HistoryAdapter. Lớp này có nhiệm vụ liên kết dữ liệu từ danh sách SensorHistory với giao diện hiển thị của từng phần tử. Thông qua onBindViewHolder, các thông tin như tên khu vực, thời gian, nhiệt độ, lửa, gas và trạng thái được gán vào các TextView tương ứng. Nhờ đó, dữ liệu được hiển thị một cách trực quan, thống nhất và dễ kiểm soát.

c. TextView và định dạng trạng thái

Trong mỗi bản ghi lịch sử, các thành phần TextView được sử dụng để hiển thị dữ liệu cảm biến dưới dạng văn bản. Ngoài việc hiển thị giá trị đo, hệ thống còn sử dụng thay đổi nền của TextView trạng thái để làm nổi bật mức độ cảnh báo. Cụ thể, khi trạng thái là cảnh báo hoặc nguy hiểm, nền hiển thị được đổi sang kiểu nổi bật; nếu ở trạng thái bình thường thì dùng nền an toàn. Cách thể hiện này giúp người dùng nhanh chóng nhận biết mức độ sự cố mà không cần đọc toàn bộ thông tin chi tiết.

d. Layout XML cho từng bản ghi

Mỗi phần tử dữ liệu lịch sử được xây dựng từ một giao diện riêng thông qua file item_history.xml. Việc sử dụng LayoutInflater để nạp giao diện cho từng dòng dữ liệu giúp bố cục hiển thị rõ ràng, đồng nhất và dễ mở rộng. Đây là cơ sở để tổ chức dữ liệu lịch sử theo dạng thẻ hoặc từng mục riêng biệt trên màn hình.

2.2.3. Ứng dụng trong hệ thống giám sát PCCC

Trong hệ thống giám sát phòng cháy chữa cháy dựa trên nền tảng IoT, các thư viện và thành phần trực quan hóa dữ liệu được ứng dụng nhằm hỗ trợ người dùng theo dõi trạng thái môi trường một cách nhanh chóng, rõ ràng và chính xác. Dữ liệu từ các cảm biến nhiệt độ, lửa, khói và khí gas tại các khu vực như phòng bếp, phòng khách và gara oto được ESP32 thu thập, sau đó truyền về ứng dụng Android thông qua giao thức MQTT. Từ nguồn dữ liệu này, ứng dụng thực hiện cập nhật giao diện theo thời gian thực và hiển thị tình trạng hoạt động của từng khu vực.

Trên giao diện chính, trạng thái của từng khu vực được trực quan hóa bằng màu sắc, nội dung và hiệu ứng hiển thị. Mỗi khu vực tương ứng với một thành

phần riêng, thể hiện các mức an toàn, cảnh báo hoặc nguy hiểm. Khi giá trị cảm biến vượt ngưỡng, giao diện sẽ thay đổi màu sắc và có thể kèm hiệu ứng nhấp nháy, giúp người dùng nhanh chóng nhận diện vị trí xảy ra sự cố mà không cần theo dõi chi tiết dữ liệu.

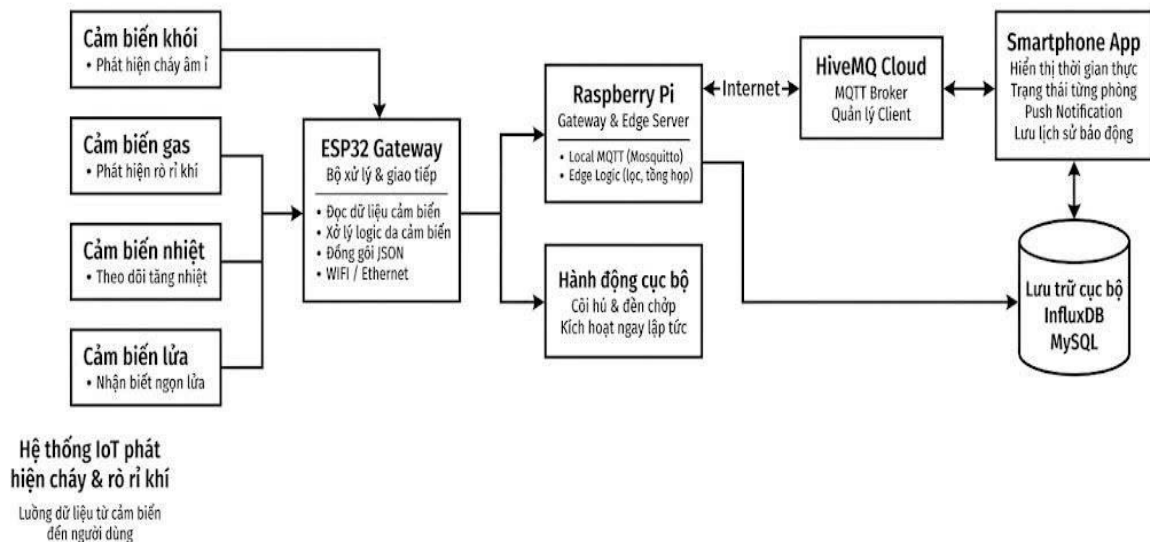
Hệ thống còn hỗ trợ lưu trữ và hiển thị lịch sử cảnh báo. Các sự kiện bất thường được ghi lại với đầy đủ thông tin như khu vực, giá trị cảm biến và thời gian, sau đó hiển thị dưới dạng danh sách thông qua RecyclerView. Việc sử dụng màu sắc để phân biệt trạng thái giúp tăng khả năng nhận diện và tra cứu thông tin.

Ngoài việc hiển thị tức thời, dữ liệu còn được trực quan hóa dưới dạng biểu đồ bằng thư viện MPAndroidChart. Điều này cho phép người dùng theo dõi xu hướng biến đổi của các thông số môi trường theo thời gian, từ đó hỗ trợ phát hiện sớm các dấu hiệu bất thường.

Như vậy, hệ thống không chỉ thực hiện chức năng giám sát mà còn cung cấp khả năng cảnh báo trực quan, lưu trữ lịch sử và hỗ trợ phân tích dữ liệu, góp phần nâng cao hiệu quả phát hiện và xử lý các nguy cơ cháy nổ.

2.3. CÁC THUẬT TOÁN CẢNH BÁO

2.3.1. Nguyên lý phát hiện cháy, khói và rò rỉ gas



Hình 2.7. Sơ đồ khối phát hiện cháy, khói và rò rỉ gas

Nguyên lý của hệ thống là giám sát đồng thời nhiều tín hiệu đặc trưng của cháy nổ, gồm khói, khí gas, nhiệt độ và ngọn lửa. Việc kết hợp nhiều cảm biến

giúp hệ thống phát hiện sớm sự cố, tăng độ chính xác và giảm báo động giả so với chỉ dùng một cảm biến đơn lẻ.

Cảm biến khói dùng để phát hiện khói ở giai đoạn đầu của đám cháy. Cảm biến gas phát hiện rò rỉ khí dễ cháy, đặc biệt hữu ích trong khu vực như nhà bếp hoặc nơi sử dụng nhiên liệu khí. Cảm biến nhiệt độ theo dõi sự tăng nhiệt bất thường, còn cảm biến lửa nhận biết trực tiếp tín hiệu ngọn lửa.

Dữ liệu từ các cảm biến sẽ được hệ thống phân tích tổng hợp. Khi chỉ một tín hiệu thay đổi nhẹ, hệ thống tiếp tục theo dõi. Khi từ hai dấu hiệu trở lên cùng vượt ngưỡng, hệ thống sẽ xác định nguy cơ cao và phát cảnh báo phù hợp. Cách tiếp cận này giúp phát hiện nhanh khói, rò rỉ gas và cháy, đồng thời nâng cao độ tin cậy khi vận hành thực tế.

2.3.2. Thuật toán phát hiện ngưỡng cảnh báo

Dựa trên dữ liệu đo từ các cảm biến, hệ thống xây dựng thuật toán phân loại trạng thái theo ba mức: Bình thường, cảnh báo và nguy hiểm. Việc phân chia thành nhiều mức giúp người dùng không chỉ nhận biết sự cố mà còn đánh giá được mức độ nghiêm trọng để có phương án xử lý phù hợp.

Trong hệ thống đã xây dựng, trạng thái được xác định dựa trên các ngưỡng của nhiệt độ, nồng độ khí gas, mật độ khói và tín hiệu lửa. Khi một hoặc nhiều thông số vượt ngưỡng an toàn, hệ thống xem đó là dấu hiệu bất thường cần theo dõi. Nếu các dấu hiệu bất thường xuất hiện đồng thời, khả năng xảy ra cháy hoặc rò rỉ khí gas sẽ cao hơn, khi đó hệ thống chuyển sang mức cảnh báo. Trong trường hợp một thông số đạt mức rất cao hoặc xuất hiện tín hiệu lửa rõ rệt, hệ thống sẽ chuyển sang mức nguy hiểm để cảnh báo khẩn cấp.

Trong quá trình thực nghiệm và xây dựng ứng dụng, đề tài sử dụng một số ngưỡng tham khảo như sau:

- Nhiệt độ cao khi lớn hơn hoặc bằng 50°C.
- Nhiệt độ rất cao khi lớn hơn hoặc bằng 70°C.
- Nồng độ khí gas cao khi giá trị cảm biến lớn hơn hoặc bằng 1200 ppm.
- Nồng độ khí gas rất cao khi giá trị cảm biến lớn hơn hoặc bằng 2500 ppm.
- Mật độ khói cao khi giá trị cảm biến vượt ngưỡng cảnh báo đã được hiệu chuẩn theo môi trường lắp đặt lớn hơn hoặc bằng 1000 ppm.
- Mật độ khói rất cao khi giá trị cảm biến vượt ngưỡng nguy hiểm đã được xác định trong quá trình hiệu chuẩn lớn hơn hoặc bằng 2000 ppm.

- Cảm biến lửa có phát hiện khi xuất hiện tín hiệu phản ánh sự hiện diện của ngọn lửa với chỉ số lớn hơn hoặc bằng 800 ADC.
- Cảm biến lửa ở mức mạnh khi giá trị đo cho thấy khả năng cháy rõ rệt với chỉ số lớn hơn hoặc bằng 2100 ADC.

Dựa trên các ngưỡng trên, thuật toán xác định trạng thái như sau:

- Trạng thái Bình thường: khi các thông số cảm biến nằm trong vùng an toàn, chưa có dấu hiệu rõ ràng của khói, rò rỉ khí gas hoặc cháy.
- Trạng thái Cảnh báo: khi xuất hiện ít nhất một yếu tố bất thường.
- Trạng thái Nguy hiểm: khi một trong các thông số đạt mức rất cao, ví dụ nhiệt độ rất cao, nồng độ khí gas rất cao, mật độ khói rất cao hoặc tín hiệu lửa ở mức mạnh.

Có thể mô tả thuật toán dưới dạng logic như sau:

- Nếu nhiệt độ $\geq$ ngưỡng rất cao
hoặc gas $\geq$ ngưỡng rất cao
hoặc khói $\geq$ ngưỡng rất cao
hoặc lửa ở mức mạnh
→ Trạng thái = Nguy hiểm.
- Ngược lại, nếu một trong các chỉ số cảm biến đạt ngưỡng cao.
→ Trạng thái = Cảnh báo.
- Trong các trường hợp còn lại:
→ Trạng thái = Bình thường.

Thuật toán này có ưu điểm là đơn giản, dễ triển khai trên cả vi điều khiển và ứng dụng di động, đồng thời phù hợp với yêu cầu xử lý thời gian thực của hệ thống IoT. Bên cạnh đó, các ngưỡng cảnh báo có thể được hiệu chỉnh theo từng không gian lắp đặt để tăng độ chính xác và hạn chế hiện tượng báo động giả trong điều kiện vận hành thực tế.

2.3.3. Phân tích dữ liệu và cảnh báo thông minh

Bên cạnh việc so sánh ngưỡng đơn giản, hệ thống còn thực hiện phân tích dữ liệu theo hướng tổng hợp và trực quan nhằm nâng cao hiệu quả cảnh báo. Dữ liệu thu được từ các nút cảm biến không chỉ được sử dụng để xác định trạng thái tức thời mà còn được truyền tới ứng dụng Android để lưu trữ, hiển thị biểu đồ và theo dõi sự biến đổi theo thời gian.

Trên ứng dụng, mỗi khu vực giám sát như bếp, phòng khách và gara được hiển thị độc lập, cho phép người dùng quan sát nhanh trạng thái của từng phòng. Dữ liệu nhận được từ giao thức MQTT được cập nhật liên tục lên giao diện, bao gồm nhiệt độ, khói, khí gas và tín hiệu lửa. Từ các giá trị này, ứng dụng tiến hành đánh giá trạng thái theo thuật toán đã xây dựng và hiển thị bằng màu sắc trực quan: màu xanh biểu thị trạng thái bình thường, màu vàng biểu thị trạng thái cảnh báo và màu đỏ biểu thị trạng thái nguy hiểm. Đối với các mức cảnh báo và nguy hiểm, giao diện còn được bổ sung hiệu ứng nhấp nháy nhằm tăng khả năng thu hút sự chú ý của người dùng.

Tính thông minh của hệ thống không chỉ nằm ở việc kiểm tra ngưỡng của từng cảm biến riêng lẻ mà còn ở khả năng kết hợp nhiều nguồn dữ liệu để đưa ra quyết định. Cách tiếp cận này giúp giảm thiểu các trường hợp báo động giả. Chẳng hạn, nếu nhiệt độ tăng do ảnh hưởng của môi trường nhưng không xuất hiện khói, không có rò rỉ khí gas và không có tín hiệu lửa thì hệ thống chưa kết luận là cháy. Ngược lại, khi nhiều yếu tố cùng xuất hiện bất thường, hệ thống sẽ nâng mức cảnh báo để phản ánh đúng mức độ nguy hiểm thực tế.

Ngoài ra, ứng dụng còn hỗ trợ lưu trữ lịch sử dữ liệu và lịch sử trạng thái cảnh báo. Nhờ đó, người dùng có thể xem lại các thời điểm xuất hiện cảnh báo, đồng thời phân tích xu hướng biến đổi của nhiệt độ, khói, khí gas và tín hiệu lửa theo thời gian. Việc lưu trữ lịch sử không chỉ phục vụ mục đích giám sát mà còn hỗ trợ quá trình kiểm tra, bảo trì và cải tiến thuật toán trong các giai đoạn tiếp theo.

Một nội dung quan trọng khác là chức năng phát hiện mất kết nối giữa ứng dụng và các nút cảm biến. Trong hệ thống, nếu sau một khoảng thời gian nhất định ứng dụng không nhận được tín hiệu trạng thái hoặc tín hiệu heartbeat từ ESP32, khu vực tương ứng sẽ được đánh dấu là mất kết nối. Cơ chế này giúp phân biệt rõ giữa trường hợp hệ thống đang ở trạng thái an toàn và trường hợp thiết bị ngừng truyền dữ liệu, từ đó tránh gây nhầm lẫn cho người sử dụng. Quá trình phân tích dữ liệu và cảnh báo thông minh trong đề tài được thực hiện theo hướng kết hợp đa cảm biến, đánh giá nhiều mức độ nguy hiểm, hiển thị trực quan theo thời gian thực và lưu trữ lịch sử để hỗ trợ giám sát lâu dài. Đây là cơ sở quan trọng giúp hệ thống hoạt động ổn định, nâng cao độ tin cậy và tăng tính ứng dụng trong thực tế.

Như vậy, chương 2 đã trình bày toàn diện các công nghệ cốt lõi cho hệ thống giám sát và cảnh báo thông minh trong hệ thống PCCC dựa trên IoT, từ

các Framework phát triển Web/Mobile đến các thư viện trực quan hóa dữ liệu và thuật toán cảnh báo thông minh. Việc lựa chọn Native Android với Kotlin và Android Studio đảm bảo hiệu suất cao, kết hợp với giao thức MQTT cho truyền dữ liệu theo thời gian thực, cùng với RecyclerView và MPAndroidChart để hiển thị trực quan. Các thuật toán đa cảm biến phân loại các trạng thái (bình thường, cảnh báo, nguy hiểm) dựa trên ngưỡng nhiệt độ, khói, gas và lửa giúp đảm bảo giảm báo động giả, tạo nền tảng vững chắc cho triển khai thực tế ở chương tiếp theo.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG PHẦN MỀM

Sau khi làm rõ cốt lõi hỗ trợ phát triển ứng dụng và xử lý dữ liệu ở chương 2, chương 3 tập trung phân tích Use Case, thiết kế giao diện người dùng và logic cảnh báo thông minh cho hệ thống giám sát PCCC. Nội dung chương này xác định rõ các chức năng chính, tương tác giữa các tác nhân (người dùng, ESP32 và cảm biến) và ứng dụng Android, đồng thời đề xuất nguyên tắc giao diện người dùng trực quan với mã màu cảnh báo (xanh - vàng - đỏ) cùng thuật toán đa ngưỡng để giảm báo động giả. Thiết kế này đảm bảo hệ thống ứng dụng hoạt động hiệu quả, thời gian thực, tạo cơ sở cho việc lập trình thực tế ở chương sau.

3.1. PHÂN TÍCH USE CASE

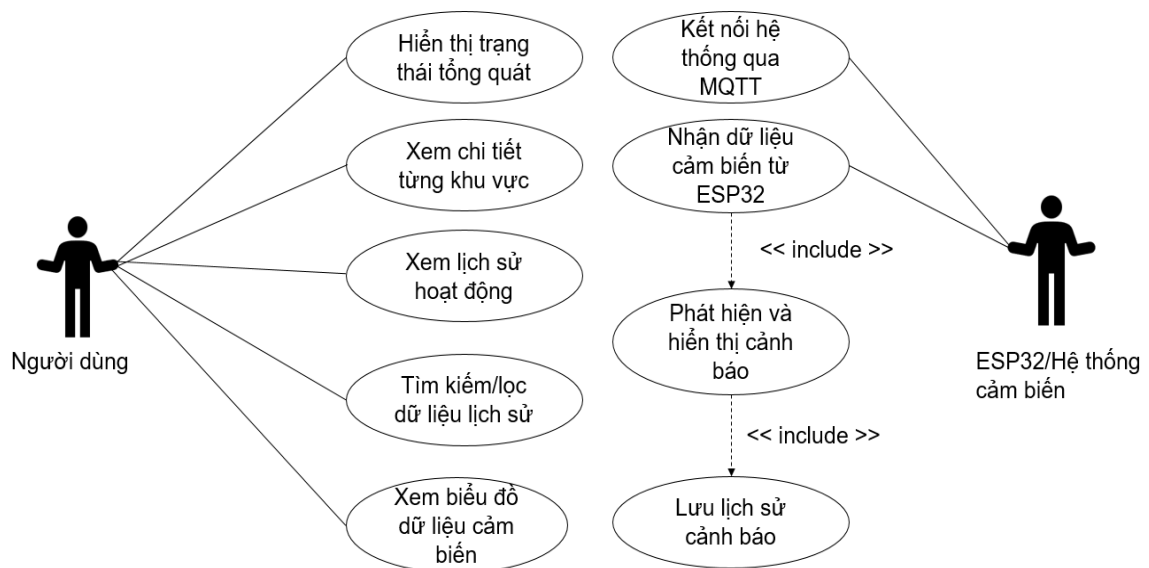
3.1.1. Xác định tác nhân hệ thống

Hệ thống có 2 tác nhân chính:

Người dùng: Là người trực tiếp sử dụng ứng dụng để giám sát hệ thống. Người dùng có thể theo dõi trạng thái tổng quan, xem chi tiết từng phòng, nhận cảnh báo khi có sự cố và tra cứu lịch sử hoạt động.

ESP32 và hệ thống cảm biến: Là tác nhân cung cấp dữ liệu đầu vào cho hệ thống. ESP32 đọc dữ liệu từ các cảm biến, xử lý sơ bộ và gửi dữ liệu lên ứng dụng thông qua MQTT để phục vụ giám sát thời gian thực.

3.1.2. Biểu đồ Use Case tổng thể



Hình 3.1. Biểu đồ Use Case tổng thể của hệ thống PCCC

3.1.3. Mô tả chi tiết các Use case

a. Use Case 1: Kết nối hệ thống qua MQTT

Tác nhân: ESP32/hệ thống cảm biến

Mô tả: Khi ứng dụng được khởi động, hệ thống tiến hành kết nối với MQTT để sẵn sàng nhận dữ liệu từ các node cảm biến

Kết quả: Ứng dụng và ESP32 thiết lập được kết nối với MQTT, hệ thống sẵn sàng cho quá trình giám sát và cập nhật dữ liệu theo thời gian thực.

b. Use Case 2: Nhận dữ liệu cảm biến từ ESP32

Tác nhân: ESP32/hệ thống cảm biến

Mô tả: ESP32 thực hiện đọc dữ liệu từ các cảm biến nhiệt độ, cảm biến lửa và cảm biến gas được lắp đặt tại từng khu vực. Sau khi thu thập dữ liệu, ESP32 xử lý sơ bộ, đóng gói giá trị đo được và gửi lên MQTT broker thông qua các topic tương ứng. Ứng dụng Android đã đăng ký sẵn các topic này sẽ nhận dữ liệu ngay khi có bản tin mới. Các giá trị nhận được sẽ được cập nhật lên giao diện, đồng thời làm dữ liệu đầu vào cho quá trình phân tích trạng thái cảnh báo của hệ thống.

Kết quả: Dữ liệu cảm biến theo môi trường tại các khu vực được truyền từ ESP32 lên ứng dụng một cách liên tục, bảo đảm khả năng giám sát theo thời gian thực.

c. Use Case 3: Hiện thị trạng thái tổng quan

Tác nhân: Người dùng

Mô tả: Tại màn hình chính, người dùng có thể quan sát trạng thái chung của toàn bộ hệ thống và trạng thái từng khu vực. Các mức trạng thái được hiển thị dưới dạng an toàn, cảnh báo hoặc nguy hiểm.

Kết quả: Người dùng nắm được tình trạng tổng quát của toàn hệ thống và xác định nhanh khu vực đang có nguy cơ xảy ra sự cố.

d. Use Case 4: Xem chi tiết từng khu vực

Tác nhân: Người dùng

Mô tả: Từ màn hình chính, người dùng có thể chọn một khu vực cụ thể như phòng bếp, phòng khách hoặc gara để xem thông tin chi tiết. Khi đó, ứng dụng sẽ chuyển sang màn hình tương ứng và hiển thị đầy đủ các thông số của khu vực được chọn, bao gồm nhiệt độ, giá trị cảm biến lửa, giá trị cảm biến gas

và trạng thái hiện tại. Dữ liệu này có thể được cập nhật liên tục theo thời gian thực để người dùng theo dõi diễn biến tại từng khu vực một cách cụ thể hơn thay vì chỉ quan sát trạng thái tổng quát.

Kết quả: Người dùng xem được các thông số chi tiết của từng khu vực, từ đó đánh giá chính xác hơn mức độ an toàn hay bất thường của khu vực đó.

e. Use Case 5: Phát hiện và hiển thị cảnh báo

Tác nhân: Ứng dụng, ESP32

Mô tả: Hệ thống phân tích dữ liệu cảm biến dựa trên các ngưỡng đã cài đặt để xác định trạng thái. Khi giá trị vượt ngưỡng, giao diện sẽ hiển thị cảnh báo thông qua màu sắc, nội dung trạng thái và hiệu ứng nhấp nháy.

Kết quả: Người dùng được cảnh báo kịp thời khi có nguy cơ cháy hoặc rò rỉ gas.

f. Use Case 6: Lưu lịch sử cảnh báo

Tác nhân: Ứng dụng

Mô tả: Khi hệ thống phát hiện trạng thái ở mức cảnh báo hoặc nguy hiểm, dữ liệu tương ứng sẽ được lưu vào cơ sở dữ liệu nội bộ, bao gồm tên khu vực, nhiệt độ, giá trị lửa, giá trị gas, trạng thái và thời gian ghi nhận.

Kết quả: Hệ thống xây dựng được cơ sở dữ liệu lịch sử cảnh báo, hỗ trợ cho việc tra cứu, đánh giá và thống kê hoạt động của hệ thống.

g. Use Case 7: Xem lịch sử hoạt động

Tác nhân: Người dùng

Mô tả: Người dùng có thể truy cập màn hình lịch sử để xem lại các sự kiện mà hệ thống đã ghi nhận trước đó. Tại đây, ứng dụng hiển thị danh sách các bản ghi theo từng thời điểm, từng khu vực và từng trạng thái cảnh báo. Mỗi bản ghi thể hiện các thông số liên quan như nhiệt độ, lửa, gas, thời gian và mức độ bất thường. Nhờ chức năng này, người dùng có thể theo dõi lại diễn biến của hệ thống trong quá khứ, kiểm tra thời điểm xuất hiện sự cố và đánh giá tình hình hoạt động của từng khu vực.

Kết quả: Người dùng tra cứu được các sự kiện đã xảy ra, hỗ trợ quá trình giám sát lâu dài và đánh giá hiệu quả vận hành của hệ thống.

h. Use Case 8: Tìm kiếm, lọc dữ liệu lịch sử

Tác nhân: Người dùng

Mô tả: Khi số lượng bản ghi lịch sử tăng lên, ứng dụng cung cấp chức năng tìm kiếm và lọc dữ liệu để giúp người dùng tra cứu thuận tiện hơn. Người dùng có thể nhập từ khóa liên quan đến tên khu vực, trạng thái hoặc chọn mốc thời gian để thu hẹp phạm vi dữ liệu cần xem. Sau khi nhận yêu cầu, hệ thống sẽ xử lý điều kiện tìm kiếm và chỉ hiển thị các bản ghi phù hợp. Chức năng này giúp việc tra cứu dữ liệu lịch sử trở nên nhanh chóng, chính xác và thuận tiện hơn trong quá trình sử dụng.

Kết quả: Người dùng tìm được các bản ghi lịch sử phù hợp với nhu cầu tra cứu mà không cần xem lại toàn bộ dữ liệu đã lưu.

i. Use Case 9: Xem biểu đồ dữ liệu cảm biến

Tác nhân : Người dùng

Mô tả: Hệ thống còn hỗ trợ hiển thị dữ liệu cảm biến dưới dạng biểu đồ, giúp người dùng dễ dàng theo dõi xu hướng thay đổi của nhiệt độ, lửa, khói và gas theo thời gian. Nhờ đó, có thể nhanh chóng phát hiện các bất thường, so sánh biến động và cảnh báo sớm nguy cơ cháy hoặc rò rỉ gas, phục vụ hiệu quả cho việc phân tích và giám sát hệ thống.

Kết quả: Người dùng theo dõi được xu hướng biến đổi của dữ liệu cảm biến một cách trực quan, từ đó nâng cao hiệu quả giám sát và khả năng phát hiện sớm bất thường.

3.2. THIẾT KẾ GIAO DIỆN NGƯỜI DÙNG

3.2.1. Nguyên tắc thiết kế UI cho hệ thống giám sát

Giao diện người dùng (UI) trong hệ thống cảnh báo cháy đóng vai trò quan trọng trong việc hỗ trợ người dùng theo dõi, phát hiện và xử lý sự cố kịp thời. Việc thiết kế UI cần đảm bảo các nguyên tắc sau:

Tính trực quan và dễ sử dụng: Giao diện cần đơn giản, rõ ràng, giúp người dùng dễ dàng quan sát trạng thái hệ thống mà không cần thao tác phức tạp.

Phản hồi nhanh theo thời gian thực: Dữ liệu từ các nút cảm biến (ESP32) được cập nhật liên tục thông qua MQTT, đảm bảo người dùng nhận được thông tin ngay khi có sự thay đổi.

Mã hóa màu sắc cảnh báo: Hệ thống sử dụng màu sắc để biểu thị mức độ nguy hiểm:

- + Xanh (SAFE): Trạng thái an toàn.
- + Vàng (WARNING): Có dấu hiệu bất thường.

+ Đỏ (DANGER): Nguy hiểm cao, cần xử lý ngay.

Tối ưu hiển thị thông tin: Màn hình chính chỉ hiển thị trạng thái tổng quát, trong khi các màn hình chi tiết (PHÒNG BẾP, PHÒNG KHÁCH, GARA) hiển thị dữ liệu cụ thể như nhiệt độ, gas, khói, lửa.

Khả năng mở rộng: Giao diện được thiết kế dạng module, cho phép dễ dàng thêm phòng hoặc cảm biến mới mà không ảnh hưởng đến hệ thống tổng thể.

3.2.2. Thiết kế giao diện Dashboard giám sát



Hình 3.2. Giao diện chính của ứng dụng

Dashboard là màn hình chính của ứng dụng, cho phép người dùng theo dõi tổng quan trạng thái của toàn bộ hệ thống.

❖ Cấu trúc giao diện:

- Tiêu đề hệ thống: “Fire Alarm System”
- Các khối hiển thị trạng thái từng phòng:
 - + PHÒNG BẾP.
 - + PHÒNG KHÁCH.
 - + GARA.

❖ **Hiện thị trạng thái:**

Mỗi phòng được biểu diễn dưới dạng một thẻ với các đặc điểm:

- + Màu nền thay đổi theo mức cảnh báo.
- + Icon minh họa
- + Trạng thái chữ (AN TOÀN/CẢNH BÁO/NGUY HIỂM).

❖ **Logic xử lý trạng thái:**

Trạng thái của mỗi phòng được xác định dựa trên các giá trị cảm biến:

- + Nhiệt độ cao.
- + Nồng độ gas.
- + Nồng độ khói.
- + Giá trị cảm biến lửa.

Ví dụ:

- AN TOÀN: Các giá trị trong ngưỡng an toàn.
- CẢNH BÁO: Một trong các giá trị vượt ngưỡng trung bình.
- NGUY HIỂM: Nhiều giá trị vượt ngưỡng nguy hiểm.

❖ **Tương tác người dùng:**

Khi nhấn vào từng phòng, hệ thống chuyển sang màn hình chi tiết, hiển thị:

- + Biểu đồ nhiệt độ theo thời gian.
- + Biểu đồ gas.
- + Giá trị lửa.

❖ **Kết nối hệ thống:**

Dashboard cũng thể hiện trạng thái kết nối:

- + ONLINE: ESP32 hoạt động bình thường.
- + OFFLINE: Mất kết nối MQTT.

3.2.3. Thiết kế giao diện cảnh báo và thông báo

Giao diện cảnh báo được thiết kế nhằm đảm bảo người dùng nhận biết sự cố một cách nhanh chóng và rõ ràng nhất.

- Cảnh báo trực quan trên giao diện:
 - + Màn hình hiển thị banner cảnh báo khi có sự cố.
 - + Hiệu ứng nhấp nháy khi ở trạng thái CẢNH BÁO hoặc NGUY HIỂM.

- + Màu sắc đỏ nổi bật cho các tình huống nguy hiểm.
- Thông báo theo thời gian thực: Khi hệ thống phát hiện cháy hoặc rò rỉ gas:
 - + Ứng dụng hiển thị thông báo ngay lập tức.
 - + Có thể tích hợp thông báo dạng push giống ứng dụng Zalo.
- Âm thanh cảnh báo:
 - + Phát âm thanh cảnh báo khi có trạng thái DANGER.
 - + Đồng bộ với còi tại nút cảm biến.
- Nội dung thông báo:

Thông báo bao gồm:

 - + Tên phòng xảy ra sự cố.
 - + Thời gian xảy ra.

Ví dụ:

“CẢNH BÁO: Phòng bếp lúc 10:25”.
- Lưu lịch sử cảnh báo: Hệ thống lưu lại các sự kiện vào cơ sở dữ liệu (Room Database), cho phép:
 - + Xem lại lịch sử.
 - + Tìm kiếm theo thời gian hoặc phòng: Ứng dụng có thể tìm kiếm theo ngày, theo giờ.
 - + Phân tích dữ liệu.



Hình 3.3. Giao diện lịch sử giám sát

3.3. XÂY DỰNG LOGIC CẢNH BÁO THÔNG MINH

3.3.1. Thu thập dữ liệu từ cảm biến

Hệ thống để thực hiện chức năng giám sát và phát hiện sớm về nguy cơ cháy nổ, hệ thống sử dụng nhiều loại cảm biến (nhiệt độ DHT22, cảm biến khí gas MQ-2, cảm biến khói, cảm biến lửa) để thu thập các thông số của môi trường thiết yếu. Mỗi cảm biến đảm nhận chức năng riêng biệt như đo nhiệt độ, phát hiện khói, nhận diện ngọn lửa và phát hiện khí gas. Các cảm biến được kết nối với vi điều khiển ESP32, sau đó gửi dữ liệu lên broker MQTT và về ứng dụng Android để xử lý tín hiệu và hiển thị lên ứng dụng.

❖ Các cảm biến chính được sử dụng trong hệ thống:

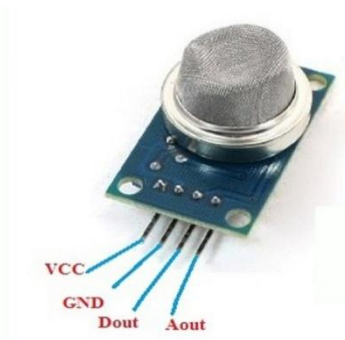
- Cảm biến nhiệt độ DHT22:



Hình 3.4. Cảm biến nhiệt độ DHT22

Cảm biến nhiệt độ DHT22 là thiết kế bán dẫn kỹ thuật số đo chính xác nhiệt độ tương đối trong khoảng -40°C đến 80°C (độ chính xác $\pm 0.5^{\circ}\text{C}$) bằng cảm biến NTC và tụ điện dung. Giúp theo dõi sự thay đổi nhiệt độ bất thường tại các khu vực nhà bếp, phòng khách và gara sẽ cảnh báo sớm khi nhiệt độ tăng nhanh hoặc vượt ngưỡng an toàn.

- Cảm biến khí gas MQ-2:



Hình 3.5. Cảm biến khí gas MQ-2

Cảm biến khí gas MQ-2 là cảm biến khí sử dụng bề mặt bán dẫn, có khả năng phát hiện các loại khí cháy như khói, khí CO, propane, một số hơi hữu cơ. Trong hệ thống PCCC, MQ-2 giúp phát hiện rò rỉ gas là một trong những nguyên nhân phổ biến gây ra cháy nổ trong khu vực nhà bếp. Cảm biến MQ-2 kết hợp với dữ liệu cảm biến nhiệt độ và khói, hệ thống có thể thông báo định mức cảnh báo hoặc nguy hiểm sớm.

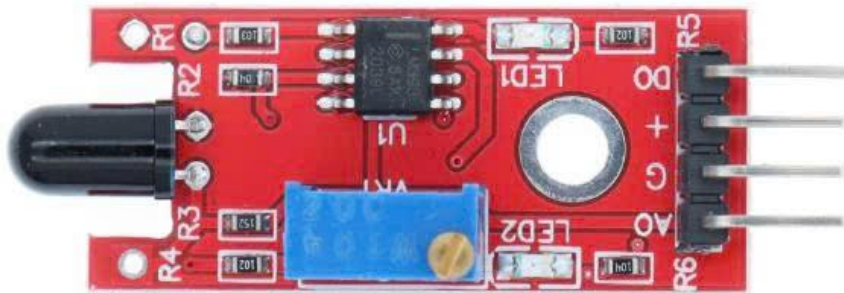
- Cảm biến khói:



Hình 3.6. Cảm biến khói MP-2

Cảm biến khói thường hoạt động dựa trên nguyên lý tán xạ ánh sáng hoặc ion hóa. Khi nồng độ khói tăng, lượng ánh sáng hoặc dòng ion đi qua buồng cảm biến bị thay đổi tạo ra tín hiệu cảnh báo. Trong hệ thống PCCC, cảm biến khói cung cấp dữ liệu về sự hiện diện của khói là dấu hiệu đầu tiên của đám cháy. Việc sử dụng cảm biến khói giúp hệ thống phát hiện được cháy ở giai đoạn đầu trước khi nhiệt độ tăng và lửa tăng quá cao.

- Cảm biến lửa:



Hình 3.7. Cảm biến lửa

Cảm biến lửa thường được hoạt động dựa trên phát hiện tia hồng ngoại IR mạnh phát ra từ ngọn lửa. Khi ngọn lửa xuất hiện cảm biến nhận tín hiệu đặc trưng từ đó gửi về hệ thống dưới dạng tín hiệu digital/analog. Cảm biến lửa giúp xác nhận giai đoạn cháy đã hình thành có ngọn lửa chứ không chỉ là khói hoặc nhiệt độ cao từ đó nâng lên mức cảnh báo.

❖ Truyền dữ liệu qua MQTT và giao tiếp với ứng dụng Android:

ESP32 đóng vai trò là node IoT thu thập dữ liệu, đóng gói và gửi dữ liệu theo các topic chuẩn như:

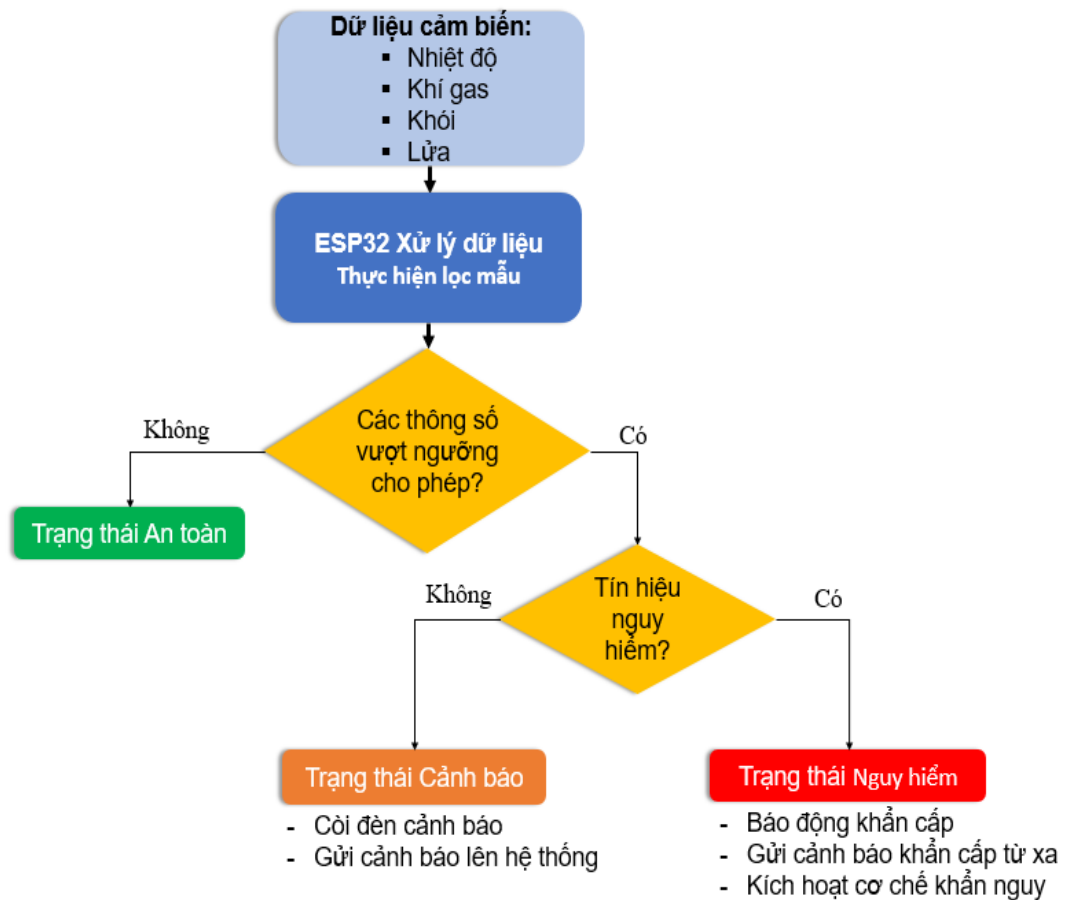
- nha/bep/nhietdo.
- nha/bep/lua.
- nha/bep/gas.
- nha/bep/khoi.
- nha/bep/trang_thai.
- nha/phong_khach/nhietdo.
- nha/phong_khach/lua.
- nha/phong_khach/khoi.
- nha/gara/nhietdo.
- nha/gara/lua.
- nha/gara/trang_thai.

Tại ứng dụng Android (ví dụ MainActivity), hệ thống khởi tạo MqttManager, kết nối với broker và đăng ký lắng nghe các topic này thông qua listener. Khi receive dữ liệu, hàm onMqttMessage được gọi, cập nhật các giá trị tương ứng vào biến trạng thái:

- kitchenTemp, kitchenFire, kitchenGas, kitchenSmoke, kitchenStatus.
- livingTemp, livingFire, livingSmoke, livingStatus.

Đồng thời, hệ thống đánh dấu thời điểm nhận tín hiệu (System.currentTimeMillis()) cho từng khu vực, từ đó xác định trạng thái online/offline dựa trên thời gian timeout (ví dụ 15 giây). Nếu không nhận tín hiệu trong khoảng thời gian quy định, hệ thống coi khu vực đó là OFFLINE và cập nhật trạng thái trên giao diện.

3.3.2. Xử lý dữ liệu và phát hiện sự cố



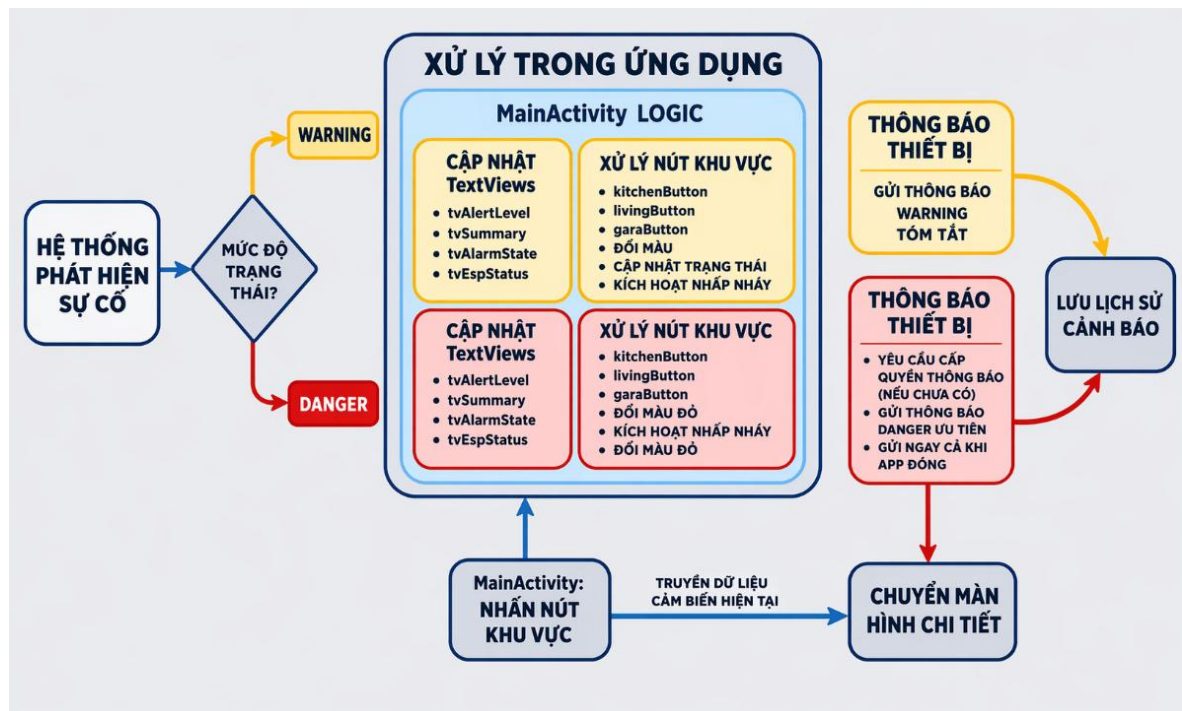
Hình 3.8. Lưu đồ xử lý dữ liệu và phát hiện sự cố

Sau khi thu thập dữ liệu từ cảm biến, hệ thống tiến hành xử lý để xác định trạng thái an toàn của từng khu vực. Dữ liệu nhiệt độ, khí gas, khói và lửa được chuẩn hóa rồi so sánh với ngưỡng an toàn đã thiết lập. Quy trình này được thiết kế để phân loại khu vực thành ba mức: An toàn, cảnh báo và nguy hiểm, giúp hệ thống nhận diện sớm nguy cơ cháy nổ và giảm cảnh báo giả.

Trong thực tế, logic xử lý được triển khai theo hướng kết hợp nhiều tín hiệu cảm biến. Nếu các thông số vẫn nằm trong giới hạn cho phép, hệ thống xác định trạng thái an toàn. Khi xuất hiện một hoặc một vài dấu hiệu bất thường như nhiệt độ tăng cao, khí gas vượt ngưỡng hoặc có tín hiệu khói, hệ thống chuyển sang mức cảnh báo để cảnh báo sớm. Khi nhiều tín hiệu nguy hiểm xuất hiện đồng thời, ví dụ nhiệt độ cao kết hợp với lửa hoặc khói, trạng thái được nâng lên nguy hiểm và kích hoạt cảnh báo khẩn cấp. Cách xử lý này phù hợp với nguyên tắc phát hiện cháy sớm: kết hợp nhiều dấu hiệu thay vì phụ thuộc vào một cảm biến đơn lẻ để tăng độ tin cậy và hạn chế cảnh báo giả.

Bên cạnh đó, hệ thống còn kiểm tra thời gian nhận tín hiệu cuối cùng của từng khu vực để xác định trạng thái ONLINE/OFFLINE. Nếu không nhận được dữ liệu trong khoảng thời gian quy định, khu vực đó được xem là mất kết nối và hiển thị trạng thái phù hợp trên giao diện giám sát. Khi trạng thái cảnh báo thay đổi, hệ thống sẽ lưu lại lịch sử sự kiện gồm tên phòng, giá trị cảm biến, trạng thái và thời điểm ghi nhận để phục vụ thống kê, trực quan hóa dữ liệu và phân tích xu hướng sau này.

3.3.3. Cơ chế gửi cảnh báo tới người dùng



Hình 3.9. Sơ đồ cơ chế cảnh báo tới người dùng

Khi hệ thống phát hiện trạng thái WARNING hoặc DANGER, ứng dụng sẽ tự động gửi cảnh báo đến người dùng thông qua giao diện chính và thông báo trên thiết bị. Trong MainActivity, các thành phần tvAlertLevel, tvSummary, tvAlarmState và tvEspStatus được cập nhật liên tục để hiển thị mức độ nguy hiểm, danh sách khu vực bị ảnh hưởng và tình trạng kết nối của các node cảm biến.

Bên cạnh đó, các nút đại diện cho từng khu vực như kitchenButton, livingButton và hallwayButton được đổi màu, cập nhật nội dung trạng thái và kích hoạt hiệu ứng nhấp nháy khi có cảnh báo. Cách hiển thị này giúp người dùng nhận biết nhanh khu vực đang có sự cố ngay từ màn hình tổng quan.

Đối với cảnh báo mức cao, hệ thống còn yêu cầu quyền thông báo trên thiết bị để có thể gửi cảnh báo ngay cả khi người dùng không mở ứng dụng. Khi nhấn vào từng khu vực, ứng dụng chuyển sang màn hình chi tiết tương ứng, truyền theo dữ liệu cảm biến hiện tại để người dùng kiểm tra và xử lý kịp thời. Ngoài ra, các sự kiện cảnh báo cũng được lưu vào lịch sử để phục vụ theo dõi và phân tích sau này.

Như vậy, Chương 3 đã tập trung phân tích và thiết kế hệ thống phần mềm cho đề tài, bao gồm cấu trúc tổng thể, chức năng của từng khối và mối liên kết giữa các thành phần trong quá trình thu thập, xử lý, truyền nhận và hiển thị dữ liệu. Trên cơ sở đó, phần mềm được xây dựng theo hướng bảo đảm khả năng giám sát thời gian thực, cảnh báo kịp thời và hỗ trợ người dùng theo dõi trạng thái hệ thống một cách trực quan. Những nội dung đã trình bày trong chương này là cơ sở quan trọng để triển khai lập trình, tích hợp hệ thống và đánh giá hiệu quả hoạt động của phần mềm trong chương tiếp theo.

CHƯƠNG 4. XÂY DỰNG ỨNG DỤNG VÀ ĐÁNH GIÁ TRẢI NGHIỆM

Sau khi hoàn thành quá trình phân tích, thiết kế và xây dựng hệ thống ở chương trước, Chương 4 tập trung trình bày kết quả triển khai thực tế của đề tài. Nội dung bao gồm việc lập trình giao diện giám sát thời gian thực trên nền tảng Android, lập trình module vẽ biểu đồ lịch sử lưu trữ và hiển thị dữ liệu lịch sử dưới dạng biểu đồ và dạng danh sách bản ghi, cũng như thiết kế và thử nghiệm cơ chế cảnh báo khi phát hiện sự cố. Đồng thời, chương này tiến hành đánh giá hiệu năng hoạt động của hệ thống thông qua các tiêu chí như độ trễ thông báo và trải nghiệm của người dùng. Các kết quả thu được từ quá trình thử nghiệm là cơ sở để nhận xét, đánh giá mức độ đáp ứng của hệ thống so với mục tiêu đề ra, đồng thời làm tiền đề cho các hướng cải tiến và phát triển trong tương lai.

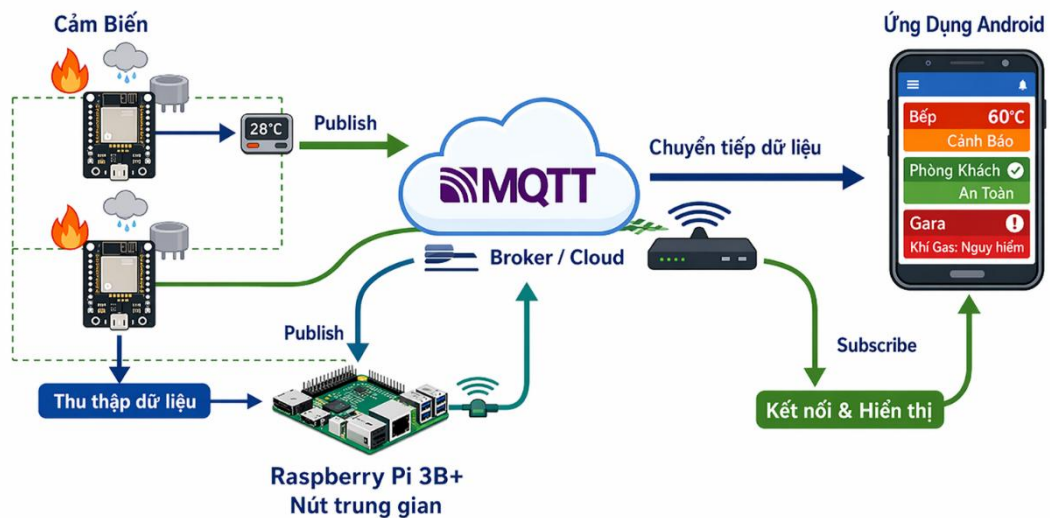
4.1. LẬP TRÌNH GIAO DIỆN GIÁM SÁT THỜI GIAN THỰC

Giao diện giám sát thời gian thực trong hệ thống PCCC được lập trình bằng Android Studio và ngôn ngữ lập trình Kotlin, sử dụng MqttService để kết nối thời gian thực với cảm biến IoT qua giao thức MQTT broker, hiển thị trạng thái sensor (nhiệt độ, khói) trên dashboard với độ trễ thấp. Dự án “appbaochay” tích hợp Socket.IO/MQTT cho push notification, hỗ trợ Activities như MainActivity, LivingRoomActivity, HistoryActivity để giám sát phòng khách, phòng bếp và gara oto.

4.1.1. Kết nối dữ liệu IoT với hệ thống

Để giao diện hoạt động theo thời gian thực, hệ thống cần có cơ chế kết nối dữ liệu ổn định giữa các nút cảm biến và ứng dụng di động. Trong mô hình này, mỗi ESP32 được bố trí tại một khu vực để thu thập dữ liệu cảm biến và gửi bản tin qua giao thức MQTT. Dữ liệu không truyền trực tiếp đến điện thoại mà được đưa về Raspberry Pi 3B+, đóng vai trò trung gian điều phối dữ liệu trong toàn hệ thống. Tại Raspberry Pi 3B+, broker MQTT được triển khai để tiếp nhận dữ liệu từ các ESP32 trong mạng nội bộ. Sau khi nhận dữ liệu, Raspberry Pi có thể quản lý kết nối, giám sát trạng thái hoạt động của các nút cảm biến, lưu trữ dữ liệu và chuyển tiếp lên broker cloud để phục vụ giám sát từ xa. Cách tổ chức này giúp hệ thống ổn định hơn và không phụ thuộc hoàn toàn vào kết nối trực tiếp giữa vi điều khiển và ứng dụng.

Luồng Dữ Liệu IoT



Hình 4.1. Sơ đồ luồng dữ liệu IoT

Ví dụ: Tại khu vực bếp, ESP32 được kết nối với cảm biến DHT22, cảm biến khí gas MQ-2, cảm biến khói MP2 và cảm biến lửa. Trong quá trình hoạt động, vi điều khiển liên tục đọc dữ liệu từ các cảm biến này. Giả sử tại một thời điểm, hệ thống ghi nhận nhiệt độ là 60°C, nồng độ khí gas tăng vượt ngưỡng cảnh báo và cảm biến lửa phát hiện tín hiệu bất thường. Khi đó, ESP32 sẽ gửi các giá trị này lên Raspberry Pi 3B+ thông qua các topic như `nha/bep/nhietdo`, `nha/bep/gas`, `nha/bep/lua` và `nha/bep/khoi`. Ứng dụng Android sau khi đã subscribe các topic tương ứng sẽ nhận được dữ liệu mới và cập nhật ngay lên giao diện. Khu vực Bếp trên màn hình lúc này sẽ chuyển sang trạng thái cảnh báo, giúp người dùng nhận biết nhanh tình huống bất thường.

Ở phía ứng dụng Android, quá trình kết nối dữ liệu trước tiên là khởi tạo MQTT client, thiết lập địa chỉ broker, cổng kết nối, tài khoản xác thực. Dưới đây là đoạn mã khởi tạo MQTT:

```
private const val TAG = "MqttManager"
// ===== MQTT SERVER =====
private const val MQTT_HOST =
    "93e32aff5d514e4c898dd92dfea0247b.s1.eu.hivemq.cloud"
private const val MQTT_PORT = 8883
private const val SERVER_URI = "ssl://$MQTT_HOST:$MQTT_PORT"
// USER / PASSWORD
private const val MQTT_USERNAME = "pi_bridge"
```

```
private const val MQTT_PASSWORD = "Pi87654321"
private val CLIENT_ID = "android-" +
  UUID.randomUUID().toString().take(6)
)
```

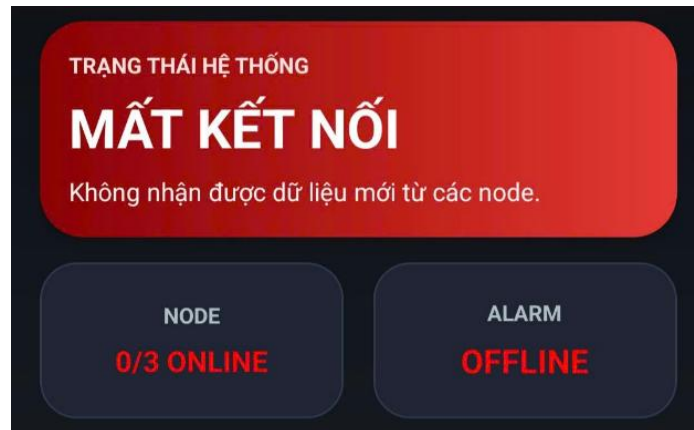
Tiếp theo sẽ thực hiện subscribe các topic cần giám sát. Khi một bản tin mới được gửi đến, callback của ứng dụng sẽ nhận dữ liệu, xác định topic nguồn và cập nhật nội dung hiển thị cho đúng khu vực tương ứng:

```
// ===== PHÒNG BẾP =====
const val TOPIC_BEP_NHIETDO = "nha/bep/nhietdo"
const val TOPIC_BEP_DOAM = "nha/bep/doam"
const val TOPIC_BEP_GAS = "nha/bep/gas"
const val TOPIC_BEP_KHOI = "nha/bep/khoi"
const val TOPIC_BEP_LUA = "nha/bep/lua"
const val TOPIC_BEP_TRANGTHAI = "nha/bep/trangthai"
const val TOPIC_BEP_CANHBAO = "nha/bep/muc_canh_bao"
const val TOPIC_BEP_MUC = TOPIC_BEP_CANHBAO
// ===== PHÒNG KHÁCH =====
const val TOPIC_PK_NHIETDO = "nha/phongkhach/nhietdo"
const val TOPIC_PK_DOAM = "nha/phongkhach/doam"
const val TOPIC_PK_GAS = "nha/phongkhach/gas"
const val TOPIC_PK_KHOI = "nha/phongkhach/khoi"
const val TOPIC_PK_LUA = "nha/phongkhach/lua"
const val TOPIC_PK_TRANGTHAI = "nha/phongkhach/trangthai"
const val TOPIC_PK_CANHBAO = "nha/phongkhach/muc_canh_bao"
const val TOPIC_PK_MUC = TOPIC_PK_CANHBAO
// ===== GARA =====
const val TOPIC_GARA_NHIETDO = "nha/gara/nhietdo"
const val TOPIC_GARA_DOAM = "nha/gara/doam"
const val TOPIC_GARA_KHOI = "nha/gara/khoi"
const val TOPIC_GARA_LUA = "nha/gara/lua"
const val TOPIC_GARA_TRANGTHAI = "nha/gara/trangthai"
const val TOPIC_GARA_CANHBAO = "nha/gara/muc_canh_bao"
```

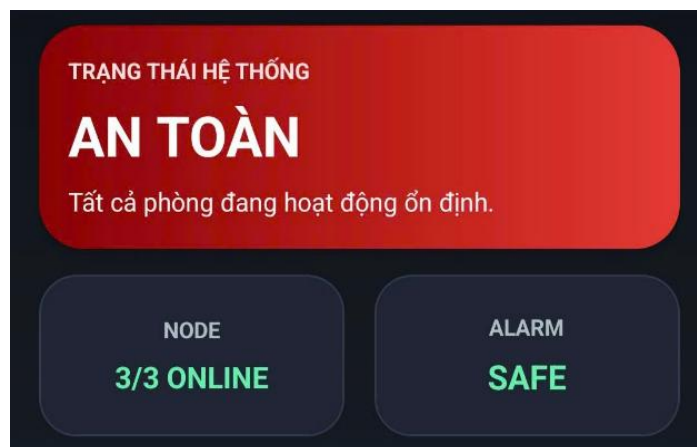
Raspberry Pi 3B+ không chỉ đảm nhiệm truyền dữ liệu mà còn giúp tăng độ ổn định cho hệ thống. Khi Internet bị gián đoạn, các nút ESP32 vẫn có thể gửi dữ liệu về Pi trong mạng nội bộ, nhờ đó hệ thống vẫn duy trì được khả năng

thu thập và quản lý dữ liệu tại chỗ. Khi kết nối được khôi phục, Pi tiếp tục đồng bộ hoặc chuyển tiếp dữ liệu, giúp giảm thiểu gián đoạn giám sát do lỗi kết nối bên ngoài.

Trên ứng dụng Android cũng cần có cơ chế kiểm tra trạng thái kết nối với broker. Khi hệ thống mất kết nối trong một khoảng thời gian nhất định, giao diện cần hiển thị rõ trạng thái ngoại tuyến thay vì giữ nguyên dữ liệu cũ. Cách xử lý này giúp tránh hiểu nhầm rằng hệ thống vẫn đang hoạt động bình thường trong khi thực tế quá trình truyền dữ liệu đã bị gián đoạn.



Hình 4.2. Giao diện hiển thị trạng thái mất kết nối MQTT



Hình 4.3. Giao diện hiển thị trạng thái khi đã kết nối với MQTT

Dữ liệu được truyền theo thời gian thực, đảm bảo độ tin cậy và tính đồng bộ giữa các thành phần. Kiến trúc này đáp ứng yêu cầu giám sát, đồng thời tạo nền tảng cho các chức năng xử lý, lưu trữ và cảnh báo trong hệ thống.

4.1.2. Hiển thị trạng thái cảm biến

Khi dữ liệu được nhận từ hệ thống MQTT thì ứng dụng Android sẽ xử lý và hiển thị trạng thái cảm biến lên giao diện. Đây là bước quan trọng vì dữ liệu thô từ cảm biến nếu chỉ hiển thị dưới dạng số đơn thuần sẽ khó giúp người dùng

đánh giá nhanh mức độ an toàn. Vậy nên trong đề tài này, dữ liệu nhận được không chỉ được hiển thị dưới dạng thông số đo mà còn được chuyển thành các mức trạng thái tổng hợp để thuận tiện cho việc theo dõi.

Trên màn hình chính mỗi khu vực giám sát được bố trí thành một thẻ thông tin riêng. Trong từng thẻ, ứng dụng hiển thị tên phòng, giá trị cảm biến hiện tại và trạng thái tổng hợp của khu vực đó. Các trạng thái được chia thành ba mức cơ bản là An toàn, Cảnh báo và Nguy hiểm. Việc chia theo ba mức giúp hệ thống không chỉ báo động khi sự cố đã xảy ra mà còn cho phép cảnh báo sớm khi xuất hiện các dấu hiệu bất thường ban đầu.



Hình 4.4. Giao diện hiển thị các trạng thái của các phòng khi gặp sự cố

Để xác định được chính xác các trạng thái này, hệ thống áp dụng các quy tắc phân loại dựa trên các ngưỡng dữ liệu cụ thể từ các cảm biến như sau:

Bảng 4.1. Quy tắc phân loại trạng thái cảm biến theo ngưỡng dữ liệu

Trạng thái	Điều kiện	Quy tắc xử lý
AN TOÀN	Nhiệt độ < 50°C; gas < 1200 ppm; khói < 1000 ppm và lửa < 800.	Trạng thái tiếp tục giám sát.
CẢNH BÁO	50°C ≤ nhiệt độ < 70°C; 1200 ppm ≤ gas < 2500 ppm; 1000 ppm ≤ khói < 2000 ppm; 800 ≤ lửa < 2100; ít nhất một cảm biến vượt ngưỡng.	Cảnh báo mức trung bình, cần theo dõi.

NGUY HIỂM	Nhiệt độ $\geq 70^{\circ}\text{C}$; gas ≥ 2500 ppm; khói ≥ 2000 ppm; lửa ≥ 2100 ; một hoặc nhiều cảm biến cùng vượt ngưỡng.	Cảnh báo mức nguy hiểm, cần xử lý ngay.
-----------	---	---

Người dùng theo dõi trực tiếp dữ liệu môi trường, ứng dụng hiển thị các thông số cảm biến của từng khu vực trên giao diện theo thời gian thực.



Hình 4.5. Giao diện hiển thị thông số cảm biến của một khu vực

Ở phía phần mềm, khi callback MQTT nhận được bản tin mới, chương trình sẽ dựa vào topic để xác định dữ liệu thuộc về phòng nào và loại cảm biến nào. Sau đó, ứng dụng cập nhật giá trị này lên các thành phần giao diện như TextView, CardView hoặc ImageView. Khi đã có đủ các dữ liệu liên quan, ứng dụng thực hiện tổng hợp và đưa ra trạng thái chung cho phòng lệnh như sau:

```
tvTemperature.text = String.format(Locale("vi", "VN"), "Nhiệt độ: %.1f °C",
temperature)
```

```
tvFire.text = String.format(Locale("vi", "VN"), "Lửa: %.0f", fireValue)
```

```
tvGas.text = String.format(Locale("vi", "VN"), "Gas: %.0f", gasValue)
```

```
tvSmoke.text = String.format(Locale("vi", "VN"), "Khói: %.0f",
smokeValue)
```

```
tvUpdatedTime.text =
```

```
    if (lastSignalTime == 0L) "Cập nhật: --"
```

```
    else "Cập nhật: ${formatTime(lastSignalTime)}"
```

```
}
```

```
private fun formatTime(time: Long): String {
```

```
    return SimpleDateFormat("dd/MM/yyyy HH:mm:ss", Locale("vi",
```

```
"VN")).format(Date(time))
}
```

Việc sử dụng màu sắc để biểu diễn trạng thái giúp tăng tính trực quan cho giao diện. Màu xanh cho biết khu vực đang hoạt động bình thường, màu vàng biểu thị dấu hiệu bất thường cần theo dõi, còn màu đỏ thể hiện nguy cơ cao cần xử lý ngay. Cách hiển thị này giúp người dùng chỉ cần quan sát tổng quát là có thể xác định nhanh khu vực nào đang có vấn đề mà không cần đọc kỹ từng chỉ số :

```
private fun updateUi() {
    val online = isOnline()
    val stateText = getRoomStateText()
    tvConnection.text = if (online) "KẾT NỐI: ONLINE" else "KẾT NỐI:
OFFLINE"
    tvConnection.setTextColor(if (online) Color.parseColor("#69F0AE") else
Color.RED)
    tvRoomState.text = "TRẠNG THÁI: $stateText"
    tvRoomState.setTextColor(
        when (stateText) {
            "NGUY HIỂM" -> Color.RED
            "CẢNH BÁO" -> Color.parseColor("#FFD54F")
            "AN TOÀN" -> Color.parseColor("#69F0AE")
            else -> Color.parseColor("#B0BEC5")
        }
    )
}
```

Tóm lại, việc hiển thị trạng thái không chỉ dừng ở mức trình bày dữ liệu mà còn bao gồm quá trình xử lý để chuyển dữ liệu đo thành thông tin có ý nghĩa, góp phần nâng cao hiệu quả giám sát và cảnh báo

4.2. LẬP TRÌNH MODULE VẼ BIỂU ĐỒ LỊCH SỬ

4.2.1. Lưu trữ dữ liệu cảm biến

Để chức năng biểu đồ lịch sử có thể hoạt động chính xác, trước hết ứng dụng cần có cơ chế lưu trữ dữ liệu cảm biến theo cấu trúc rõ ràng và đồng bộ theo thời gian. Dữ liệu từ các node cảm biến được gửi về ứng dụng Android thông qua giao thức MQTT. Sau khi tiếp nhận bản tin, chương trình tiến hành phân tích topic để xác định đúng khu vực giám sát và loại dữ liệu tương ứng, sau đó đóng gói thành bản ghi hoàn chỉnh và lưu vào cơ sở dữ liệu cục bộ. Từ

đó giúp dữ liệu lịch sử được lưu giữ liên tục, tạo nền tảng cho việc truy vấn và hiển thị lại diễn biến môi trường dưới dạng biểu đồ theo thời gian.

Trong môi trường Android, nhóm sử dụng thư viện Room để xây dựng cơ sở dữ liệu cục bộ. Việc sử dụng Room giúp cho quá trình lưu trữ và truy xuất dữ liệu thuận tiện hơn, đồng thời bảo đảm dữ liệu được tổ chức có cấu trúc. Mỗi bản ghi lịch sử bao gồm các trường như tên khu vực, nhiệt độ, giá trị lửa, nồng độ khí gas, nồng độ khói, trạng thái cảnh báo và thời điểm ghi nhận. Trong đó, trường thời gian có vai trò đặc biệt quan trọng, vì đây là căn cứ để sắp xếp dữ liệu theo đúng thứ tự diễn biến thực tế khi hiển thị biểu đồ. Nội dung này được thể hiện thông qua entity `SensorHistory` trong mã nguồn ứng dụng:

```
@Entity(tableName = "sensor_history")
data class SensorHistory(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    val roomName: String,
    val temperature: Float,
    val fireValue: Float,
    val gasValue: Float,
    val smokeValue: Float,
    val status: String,
    val timestamp: String
)
```

Bên cạnh entity lưu trữ bản ghi, nhóm còn xây dựng lớp cơ sở dữ liệu cục bộ `AppDatabase` để quản lý việc tạo bảng, khởi tạo cơ sở dữ liệu và cung cấp các DAO phục vụ truy xuất dữ liệu trong ứng dụng. Việc sử dụng cơ sở dữ liệu cục bộ giúp ứng dụng vẫn có thể lưu giữ lịch sử hoạt động trong quá trình giám sát, đồng thời tạo điều kiện để người dùng xem lại dữ liệu theo từng thời điểm, ngay cả khi không cần truy cập trực tiếp đến dữ liệu thô bên trong hệ quản trị. Cấu trúc này đã được nhóm xây dựng trong mã nguồn như sau:

```
@Database(
    entities = [
        SensorHistory::class,
        RoomConfig::class
    ],
    version = 3,
```

```

    exportSchema = false
)
abstract class AppDatabase : RoomDatabase() {
    abstract fun sensorHistoryDao(): SensorHistoryDao
    abstract fun roomConfigDao(): RoomConfigDao
    companion object {
        @Volatile
        private var INSTANCE: AppDatabase? = null
        fun getDatabase(context: Context): AppDatabase {
            return INSTANCE ?: synchronized(this) {
                val instance = Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "fire_alarm_db"
                )
                    .fallbackToDestructiveMigration()
                    .build()
                INSTANCE = instance
            }
        }
    }
}

```

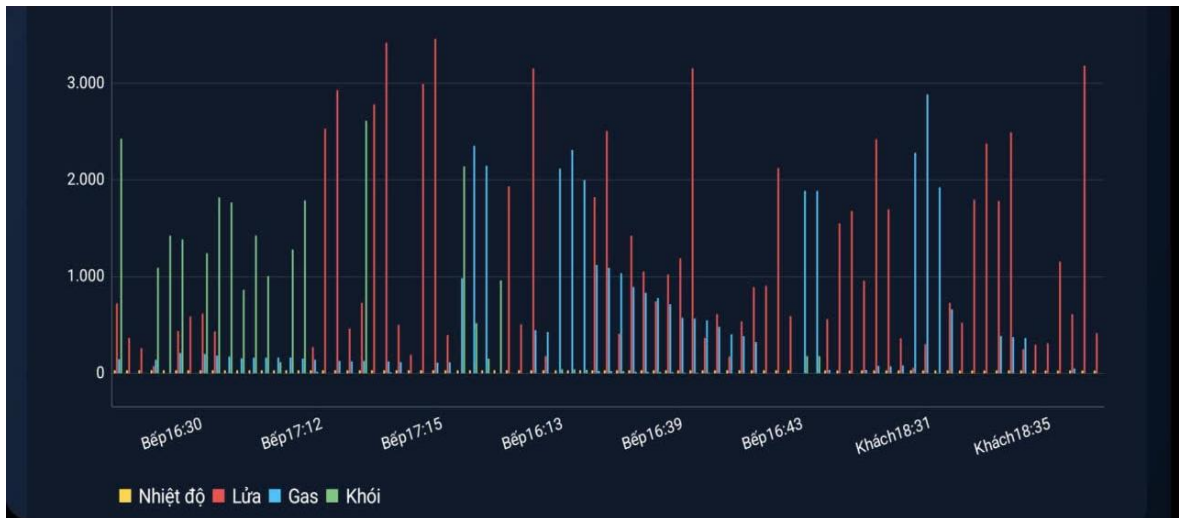
Từ góc độ thực tiễn, cơ chế lưu trữ dữ liệu bằng Room cho phép ứng dụng duy trì lịch sử biến đổi của môi trường theo từng phòng và theo từng mốc thời gian. Đây là cơ sở quan trọng để module biểu đồ lịch sử có thể trực quan hóa lại quá trình thay đổi của nhiệt độ, khói, gas và lửa. Có thể nói, nếu không có bước lưu trữ dữ liệu có cấu trúc thì hệ thống chỉ dừng ở mức hiển thị trạng thái tức thời, chưa hỗ trợ được việc theo dõi xu hướng biến đổi theo thời gian. Vì vậy, phần lưu trữ dữ liệu cảm biến chính là nền tảng đầu vào cho toàn bộ chức năng trực quan hóa lịch sử trên ứng dụng.

4.2.2. Hiển thị biểu đồ dữ liệu theo thời gian

Sau khi dữ liệu đã được lưu vào cơ sở dữ liệu cục bộ, bước tiếp theo là truy xuất và hiển thị lại dữ liệu đó dưới dạng biểu đồ theo thời gian. Đây là chức năng có ý nghĩa thực tế rõ rệt, người dùng không chỉ xem được giá trị hiện

tại của cảm biến mà còn theo dõi được xu hướng biến đổi của môi trường trong một khoảng thời gian nhất định. Thông qua biểu đồ, người dùng có thể phát hiện sớm những dấu hiệu bất thường như nhiệt độ tăng nhanh, nồng độ khói tăng đột biến hoặc nhiều thông số cùng biến động trước khi hệ thống chuyển sang trạng thái nguy hiểm.

Nhóm sử dụng thư viện MPAndroidChart nhằm trực quan hóa dữ liệu lịch sử trên Android. Dữ liệu lấy từ Room được sắp xếp theo thời gian, sau đó chuyển đổi sang tập điểm dữ liệu phù hợp với cấu trúc của thư viện biểu đồ. Trong biểu đồ, trục hoành biểu diễn thời gian và trục tung biểu diễn giá trị của các thông số cảm biến như nhiệt độ, lửa, gas và khói. Dạng biểu đồ cột được lựa chọn vì phù hợp với cách hiển thị nhiều thông số trong cùng một khoảng thời gian, đồng thời giúp người dùng dễ quan sát và so sánh sự thay đổi của các giá trị tại từng mốc thời gian.



Hình 4.6. Giao diện biểu đồ lịch sử dạng cột

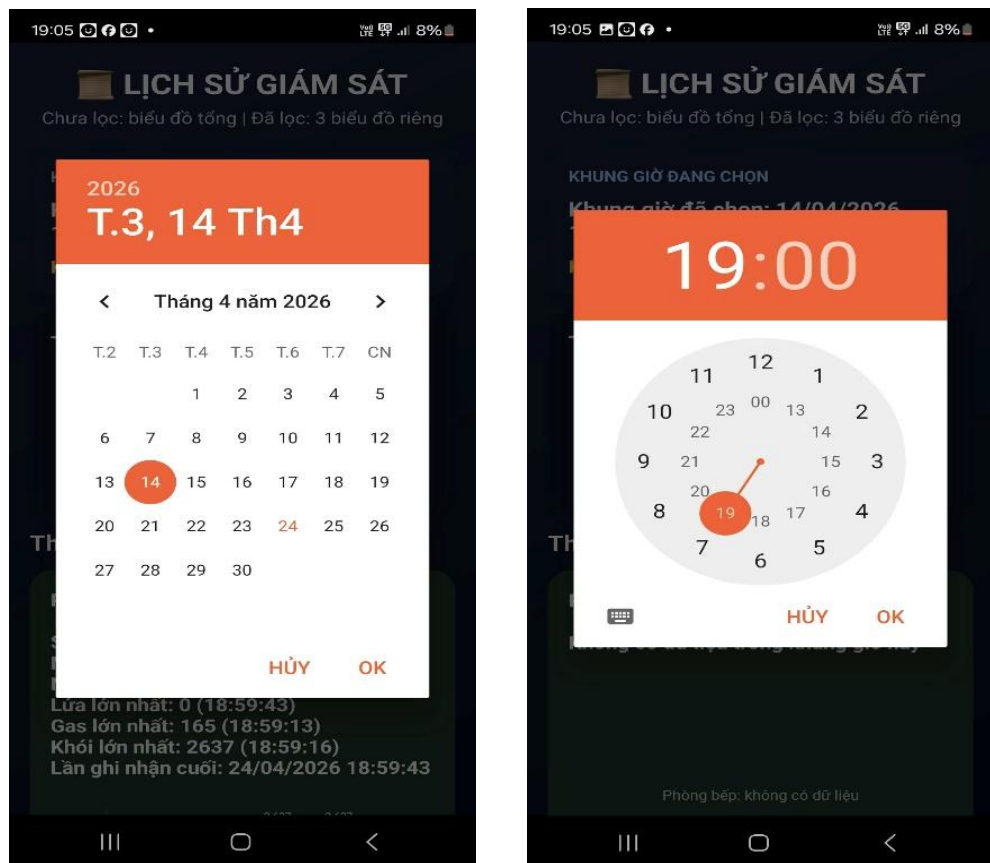
Để biểu đồ hiển thị đúng dữ liệu cần thiết, trước tiên ứng dụng thực hiện truy vấn dữ liệu lịch sử theo khung thời gian hoặc theo điều kiện người dùng lựa chọn. Trong mã nguồn hiện tại, quá trình này được thực hiện qua hàm `performTimeSearch()`. Khi người dùng nhập mốc thời gian cần xem, hệ thống sẽ truy vấn các bản ghi phù hợp trong cơ sở dữ liệu và trả kết quả về giao diện để tiếp tục xử lý. Đây là bước trung gian quan trọng giữa cơ sở dữ liệu và biểu đồ:

```
private fun performTimeSearch() {
    lifecycleScope.launch(Dispatchers.IO) {
        val result = dao.searchByDateTime(selectedDateTimePrefix)
```

```

withContext(Dispatchers.Main) {
    updateFilteredUI(result)
    if (result.isEmpty()) {
        Toast.makeText(
            this@HistoryActivity,
            "Khung giờ này không có dữ liệu",
            Toast.LENGTH_SHORT
        ).show()
    }
}
}
}

```



Hình 4.7. Giao diện lọc dữ liệu lịch sử theo khung thời gian trên ứng dụng

Giao diện sau khi người dùng chọn một khoảng thời gian cụ thể. Mục đích là chứng minh chức năng tìm kiếm theo thời gian hoạt động đúng, đồng thời cho thấy biểu đồ không chỉ hiển thị toàn bộ lịch sử mà còn hỗ trợ lọc dữ liệu để người dùng tập trung vào giai đoạn cần theo dõi.



Hình 4.8. Giao diện lịch sử khi không có dữ liệu



Hình 4.9. Giao diện lịch sử khi truy xuất ra dữ liệu

Sau khi truy vấn, dữ liệu tiếp tục được chuyển đổi từ danh sách bản ghi sang cấu trúc dữ liệu của thư viện biểu đồ. Mỗi bản ghi được ánh xạ thành một phần tử BarEntry, trong đó trục X là vị trí của thời điểm ghi nhận và trục Y là

giá trị cảm biến tương ứng. Đồng thời, hệ thống tạo riêng các tập dữ liệu cho nhiệt độ, lửa, gas và khói để thuận lợi trong việc hiển thị và so sánh. Quá trình này được nhóm triển khai trong hàm `updateSingleRoomChart()`:

```
private fun updateSingleRoomChart(
    chart: BarChart,
    roomData: List<SensorHistory>,
    emptyText: String
){
    if (roomData.isEmpty()) {
        roomChartDataMap[chart.id] = emptyList()
        chart.clear()
        chart.setNoDataText(emptyText)
        chart.invalidate()
        return
    }
    val reversedData = roomData.reversed()
    roomChartDataMap[chart.id] = reversedData
    val tempEntries = ArrayList<BarEntry>()
    val fireEntries = ArrayList<BarEntry>()
    val gasEntries = ArrayList<BarEntry>()
    val smokeEntries = ArrayList<BarEntry>()
    val labels = ArrayList<String>()

    reversedData.forEachIndexed { index, item ->
        tempEntries.add(BarEntry(index.toFloat(), item.temperature))
        fireEntries.add(BarEntry(index.toFloat(), item.fireValue))
        gasEntries.add(BarEntry(index.toFloat(), item.gasValue))
        smokeEntries.add(BarEntry(index.toFloat(), item.smokeValue))
        labels.add(shortTime(item.timestamp))
    }
}
```

Sau khi tạo được các tập dữ liệu, ứng dụng tiếp tục cấu hình biểu đồ thông qua các `BarDataSet`. Mỗi thông số được gán một màu riêng để người dùng dễ phân biệt khi quan sát. Cách hiển thị này không chỉ mang tính trực quan mà còn hỗ trợ tốt trong quá trình phân tích xu hướng biến đổi của môi trường theo thời gian. Phần này được thể hiện như sau:

```

(tempEntries, "Nhiệt độ").apply {
    color = Color.parseColor("#FFD54F")
    valueTextColor = Color.WHITE
    valueTextSize = 8.5f
}
val fireSet = BarDataSet(fireEntries, "Lửa").apply {
    color = Color.parseColor("#EF5350")
    valueTextColor = Color.WHITE
    valueTextSize = 8.5f
}
val gasSet = BarDataSet(gasEntries, "Gas").apply {
    color = Color.parseColor("#4FC3F7")
    valueTextColor = Color.WHITE
    valueTextSize = 8.5f
}
val smokeSet = BarDataSet(smokeEntries, "Khói").apply {
    color = Color.parseColor("#81C784")
    valueTextColor = Color.WHITE
    valueTextSize = 8.5f
}

```



Hình 4.10. Biểu đồ tổng quan bất thường của các phòng

4.3. DEMO TÍNH NĂNG CẢNH BÁO QUA ĐIỆN THOẠI KHI CÓ SỰ CỐ

4.3.1. Cơ chế gửi thông báo

Cơ chế gửi thông báo được xây dựng dựa trên luồng xử lý dữ liệu từ các node ESP32 thu thập dữ liệu từ cảm biến tại từng khu vực như bếp, phòng khách và gara, sau đó gửi dữ liệu về Raspberry Pi thông qua giao thức MQTT nội bộ. Raspberry Pi đóng vai trò trung gian, thực hiện tiếp nhận dữ liệu, lưu trữ vào cơ sở dữ liệu cục bộ và chuyển tiếp bản tin lên MQTT Broker trên nền tảng cloud. Ứng dụng Android sau khi kết nối tới MQTT Broker sẽ đăng ký các topic tương ứng với từng khu vực giám sát. Khi có bản tin mới, ứng dụng tiến hành đọc dữ liệu cảm biến, đối chiếu với ngưỡng phân loại trạng của từng khu vực theo ba mức: SAFE, WARNING và DANGER.

Cơ chế phản hồi của ứng dụng được thực hiện như sau:

- Nếu trạng thái là SAFE, ứng dụng chỉ cập nhật thông số cảm biến và giao diện hiển thị.
- Nếu trạng thái chuyển sang WARNING, ứng dụng chuyển giao diện sang trạng thái cảnh báo và tạo thông báo trên điện thoại.
- Nếu trạng thái chuyển sang DANGER, ứng dụng chuyển giao diện sang trạng thái nguy hiểm, đồng thời tạo thông báo mức ưu tiên cao để người dùng kịp thời xử lý.

Để đảm bảo hệ thống vẫn có khả năng cảnh báo khi người dùng không mở trực tiếp ứng dụng, ứng dụng Android sử dụng cơ chế chạy nền bằng Foreground Service kết hợp với Notification Channel. Foreground Service giúp duy trì quá trình kết nối và lắng nghe dữ liệu MQTT trong nền, còn Notification Channel được dùng để phân loại mức độ ưu tiên của thông báo nhằm cho phép điện thoại rung, phát âm báo và hiển thị thông báo nổi khi có sự cố.

Để triển khai cơ chế này trên Android, trước hết cần khởi tạo kênh thông báo để quản lý các loại cảnh báo khác nhau:

```
private fun createChannels() {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        val monitor = NotificationChannel(
            CHANNEL_MONITOR,
            "Giám sát hệ thống",
            NotificationManager.IMPORTANCE_LOW
        )
    }
}
```

```

val alert = NotificationChannel(
    CHANNEL_ALERT,
    "Cảnh báo cháy",
    NotificationManager.IMPORTANCE_HIGH
).apply {
    description = "Thông báo cảnh báo cháy, khói, nhiệt độ, rò rỉ gas"
    enableVibration(true)
    enableLights(true)
    lightColor = Color.RED
}
val manager = getSystemService(NotificationManager::class.java)
manager.createNotificationChannel(monitor)
manager.createNotificationChannel(alert)
}
}
}

```

Trong đoạn mã CHANNEL_MONITOR được sử dụng cho thông báo duy trì dịch vụ giám sát nền, còn CHANNEL_ALERT được sử dụng cho các cảnh báo dữ liệu cảm biến. Việc thiết lập IMPORTANCE_HIGH, kết hợp với rung và đèn báo, giúp thông báo cảnh báo có mức độ ưu tiên cao hơn so với thông báo thông thường.

Khi nhận dữ liệu từ MQTT ứng dụng kiểm tra trạng thái mới của khu vực và so sánh với trạng thái trước đó. Nếu trạng thái không thay đổi, hệ thống không tạo thêm thông báo mới. Ngược lại, nếu trạng thái chuyển từ SAFE sang WARNING hoặc DANGER, ứng dụng sẽ tạo thông báo tương ứng. Đoạn mã xử lý thông báo khi trạng thái thay đổi được trình bày như sau:

```

private fun notifyIfChanged(
    state: String,
    lastState: String,
    room: String,
    idBase: Int,
    update: (String) -> Unit
){
    if (state == lastState) return
    when (state) {

```

```

"WARNING" -> {
    showAlert(
        title = "⚠️ CẢNH BÁO - $room",
        msg = "$room đang ở mức cảnh báo. Hãy kiểm tra ngay.",
        id = idBase
    )
    update("WARNING")
}
"DANGER" -> {
    showAlert(
        title = "🔥 NGUY HIỂM - $room",
        msg = "$room đang ở mức nguy hiểm. Cần xử lý ngay lập tức.",
        id = idBase + 1
    )
    update("DANGER")
}
"SAFE" -> update("")
}
}

```

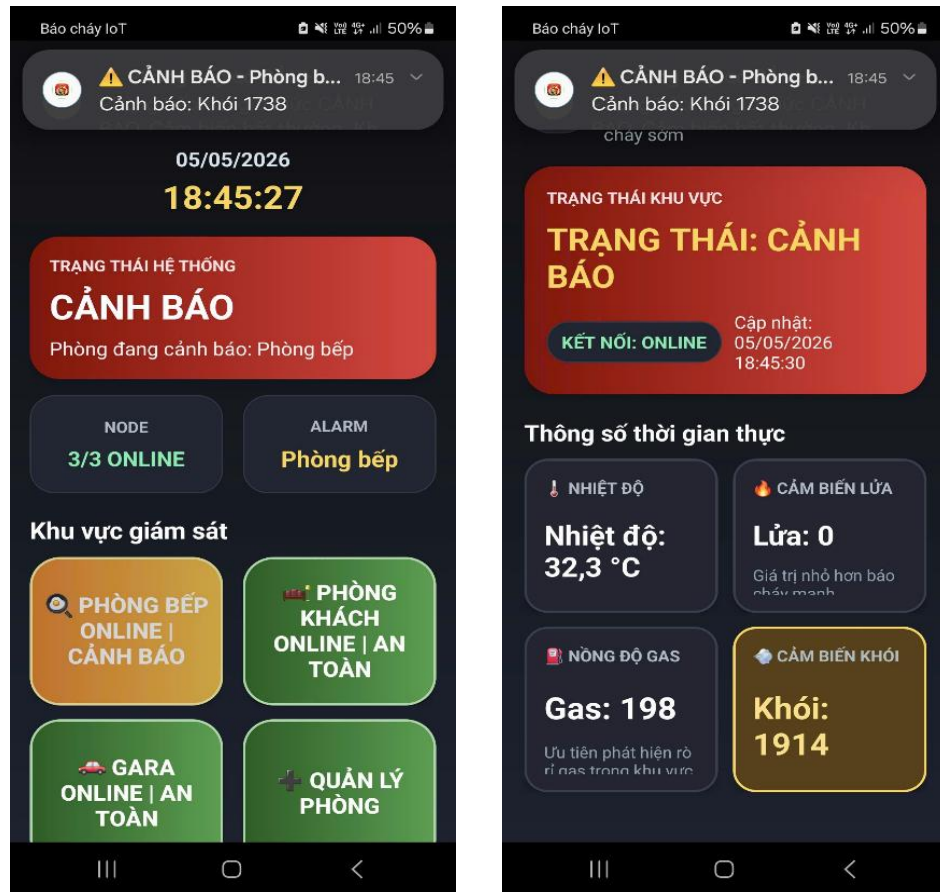
Như vậy, cơ chế trên giúp hệ thống chỉ gửi thông báo khi trạng thái cảnh báo có sự thay đổi, tránh tình trạng gửi lặp lại nhiều thông báo giống nhau trong khi sự cố vẫn đang duy trì. Nhờ đó, ứng dụng vừa đảm bảo cảnh báo kịp thời, vừa hạn chế gây nhiễu cho người dùng.

4.3.2. Thử nghiệm hệ thống cảnh báo

Mục đích của thử nghiệm này là đánh giá khả năng phản hồi của ứng dụng Android khi hệ thống phát hiện sự cố tại các khu vực giám sát. Nội dung thử nghiệm tập trung vào việc kiểm chứng thuật toán phân loại trạng thái theo ngưỡng cảnh báo và đánh giá cơ chế gửi thông báo đến điện thoại khi ứng dụng hoạt động ở chế độ nền.

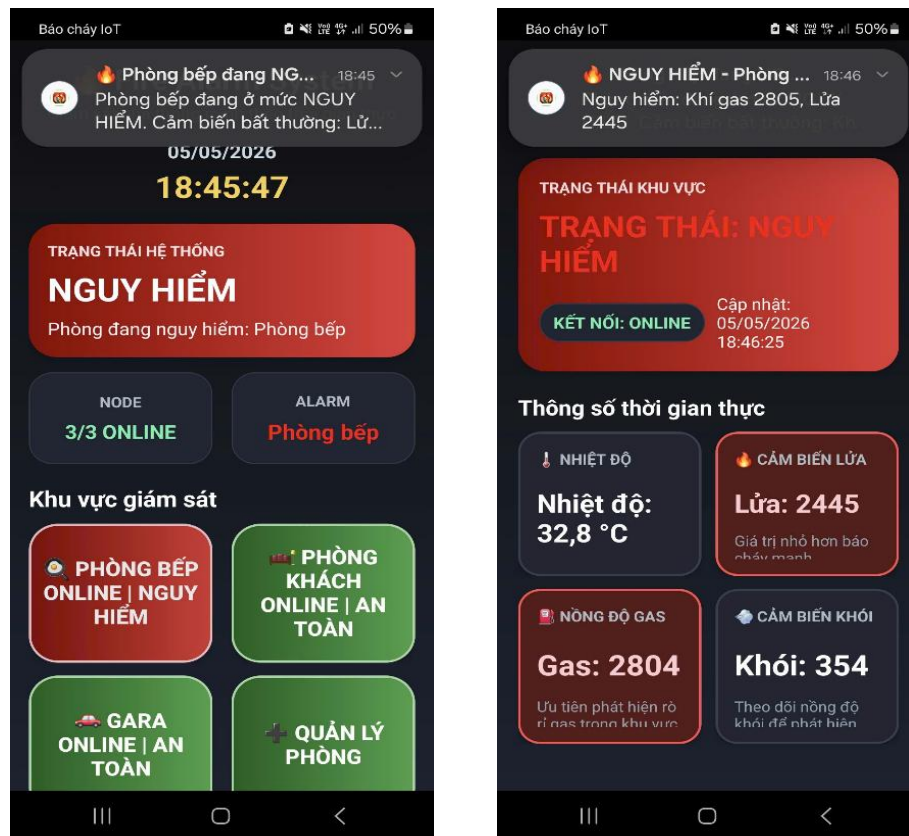
Trong quá trình thử nghiệm, nhóm tạo các tình huống như nhiệt độ tăng cao, xuất hiện khói, khí gas hoặc tín hiệu lửa tại các node cảm biến. Dữ liệu sau khi được ESP32 thu thập sẽ được gửi lên hệ thống thông qua giao thức MQTT. Ứng dụng Android tiếp nhận bản tin, xử lý trạng thái và cập nhật cảnh báo tương ứng trên giao diện.

Trong hình ứng dụng đã nhận đúng dữ liệu từ hệ thống MQTT và chuyển trạng thái khu vực bếp sang mức cảnh báo. Việc sử dụng màu vàng giúp người dùng nhận biết nhanh tình huống bất thường nhưng chưa đạt mức nguy hiểm cao.



Hình 4.11. Giao diện ứng dụng khi phòng bếp chuyển mức cảnh báo

Trường hợp giá trị cảm biến vượt ngưỡng nguy hiểm. Khi đó, giao diện chuyển sang màu đỏ và hiển thị cảnh báo rõ ràng, giúp người dùng nhanh chóng nhận biết mức độ nghiêm trọng của sự cố.



Hình 4.13. Giao diện ứng dụng khi phòng bếp chuyển mức nguy hiểm

Bên cạnh việc hiển thị cảnh báo trên giao diện chính, nhóm cũng thử nghiệm cơ chế gửi thông báo khi ứng dụng không được mở trực tiếp trên màn hình. Trong trường hợp điện thoại đang khóa màn hình hoặc đang mở ứng dụng khác, ứng dụng giám sát vẫn có thể nhận dữ liệu cảnh báo thông qua cơ chế chạy nền.



Hình 4.12. Thông báo cảnh báo hiển thị khi không mở trong App

Kết quả thử nghiệm cho thấy thông báo vẫn được gửi đến điện thoại ngay cả khi người dùng không mở ứng dụng. Điều này chứng minh cơ chế chạy nền

và gửi notification hoạt động đúng yêu cầu, phù hợp với mục tiêu cảnh báo từ xa của hệ thống. Các cảnh báo vẫn được hiển thị khi ứng dụng hoạt động nền hoặc khi điện thoại đang khóa màn hình, góp phần nâng cao khả năng giám sát và hỗ trợ người dùng phát hiện sự cố kịp thời.

4.4. ĐÁNH GIÁ TRẢI NGHIỆM NGƯỜI DÙNG VÀ ĐỘ TRỄ CỦA THÔNG BÁO

4.4.1. Phương pháp đánh giá

Độ trễ thông báo được xác định là khoảng thời gian từ khi node ESP32 phát hiện sự cố và kích hoạt cảnh báo tại chỗ đến khi điện thoại nhận được thông báo cảnh báo. Khoảng thời gian này phản ánh khả năng đáp ứng gần thời gian thực của toàn bộ hệ thống, bao gồm quá trình đọc cảm biến, xử lý dữ liệu, truyền bản tin MQTT, nhận dữ liệu trên ứng dụng Android và kích hoạt notification.

Trong quá trình thử nghiệm, nhóm thực hiện đo độ trễ bằng phương pháp quan sát thực nghiệm kết hợp đồng hồ bấm giờ.

Quy trình đo được thực hiện như sau:

- Bước 1: Khởi động hệ thống và kiểm tra kết nối giữa ESP32, Raspberry Pi, MQTT broker và ứng dụng Android.
- Bước 2: Đặt điện thoại ở các trạng thái khác nhau như đang mở ứng dụng, đang mở ứng dụng khác hoặc đang khóa màn hình.
- Bước 3: Tạo tác nhân giả lập sự cố tại node cảm biến, ví dụ tăng nhiệt độ, tạo khói hoặc đưa nguồn lửa đến gần cảm biến.
- Bước 4: Khi LED và còi cảnh báo tại node bắt đầu hoạt động, ghi nhận thời điểm bắt đầu bằng đồng hồ bấm giờ.
- Bước 5: Khi điện thoại hiển thị notification, rung hoặc phát âm thanh cảnh báo, ghi nhận thời điểm kết thúc.

Thử nghiệm được thực hiện nhiều lần nhằm bảo đảm kết quả có tính ổn định. Nhóm tiến hành đo trong hai điều kiện mạng chính: điện thoại và hệ thống cùng sử dụng mạng Wi-Fi và điện thoại sử dụng mạng di động 5G. Mỗi điều kiện được thử nghiệm ở các trạng thái khác nhau của điện thoại để đánh giá độ trễ trong các tình huống sử dụng thực tế.

Bảng 4.2. Bảng đo độ trễ thông báo

STT	Điều kiện	Trạng thái điện thoại	Số lần đo	Mốc bắt đầu	Mốc kết thúc
1	Wi-Fi	Ứng dụng đang mở	5	Led và còi tại node bật	Thông báo xuất hiện
2	Wi-Fi	Ứng dụng chạy nền	5	Led và còi tại node bật	Thông báo xuất hiện
3	Wi-Fi	Khóa màn hình	5	Led và còi tại node bật	Thông báo xuất hiện
4	5G	Ứng dụng đang mở	5	Led và còi tại node bật	Thông báo xuất hiện
5	5G	Ứng dụng chạy nền	5	Led và còi tại node bật	Thông báo xuất hiện
6	5G	Khóa màn hình	5	Led và còi tại node bật	Thông báo xuất hiện

Tổng số lần thử nghiệm là 30 lần, các kết quả chi tiết của từng lần đo được lưu lại để kiểm tra. Do phép đo được thực hiện thủ công bằng đồng hồ bấm giờ nên kết quả có thể tồn tại sai số nhỏ do phản xạ của người đo và thời điểm quan sát thông báo. Vậy nên, nhóm thực hiện lặp lại nhiều lần trong từng điều kiện thử nghiệm và sử dụng giá trị trung bình để giảm ảnh hưởng của sai số đơn lẻ.

Bên cạnh việc đo độ trễ thông báo, nhóm tiến hành đánh giá trải nghiệm người dùng nhằm kiểm tra mức độ dễ sử dụng, tính trực quan và khả năng hỗ trợ cảnh báo của ứng dụng Android. Nội dung đánh giá tập trung vào giao diện Dashboard, khả năng nhận biết trạng thái cảnh báo, mức độ rõ ràng của thông báo và sự thuận tiện khi tra cứu dữ liệu lịch sử.

Đối tượng tham gia thử nghiệm gồm 15 người dùng không chuyên, tức là những người không trực tiếp tham gia thiết kế hoặc lập trình hệ thống. Người dùng được yêu cầu thao tác với các chức năng chính của ứng dụng như quan sát trạng thái các khu vực trên Dashboard, mở màn hình chi tiết từng phòng, nhận biết trạng thái cảnh báo thông qua màu sắc, kiểm tra thông báo khi có sự cố và tra cứu dữ liệu lịch sử bằng biểu đồ.

Người dùng đánh giá từng tiêu chí theo ba mức: Tốt, chấp nhận được và cần cải thiện. Kết quả được tổng hợp theo tỷ lệ phần trăm để phục vụ quá trình phân tích và nhận xét.

Bảng 4.3. Tiêu chí đánh giá trải nghiệm người dùng

STT	Tiêu chí đánh giá	Nội dung đánh giá
1	Dashboard	Mức độ dễ quan sát trạng thái từng khu vực
2	Màu sắc cảnh báo	Khả năng nhận biết trạng thái xanh, vàng, đỏ
3	Notification	Mức độ rõ ràng của khu vực và mức cảnh báo
4	Thông số cảm biến	Mức độ dễ đọc, dễ hiểu của dữ liệu cảm biến
5	Biểu đồ lịch sử	Mức độ dễ đọc, dễ hiểu của dữ liệu cảm biến
6	Thao tác cảnh báo	Khả năng xác định nhanh khu vực xảy ra sự cố

4.4.2. Kết quả thử nghiệm

Sau khi thực hiện 30 lần thử nghiệm ở các điều kiện khác nhau, nhóm tổng hợp kết quả độ trễ thông báo như bảng 4.4. Các giá trị được trình bày gồm độ trễ nhỏ nhất, độ trễ lớn nhất và độ trễ trung bình.

Bảng 4.4. Kết quả tổng hợp độ trễ thông báo

STT	Điều kiện thử nghiệm	Số lần đo	Độ trễ nhỏ nhất	Độ trễ trung bình	Độ trễ lớn nhất
1	Wi-Fi, ứng dụng đang mở	5	1,13s	1,75s	2,56s
2	Wi-Fi, ứng dụng chạy nền	5	2,23s	3,28s	3,92s
3	Wi-Fi, khóa màn hình	5	2,51s	3,23s	4,05s
4	5G, ứng dụng đang mở	5	0,91s	1,72s	2,16s
5	5G, ứng dụng chạy nền	5	1,51s	2,24s	3,11s

6	5G, khóa màn hình	5	1,79s	2,63s	3,05s
---	-------------------	---	-------	-------	-------

Từ kết quả trên, có thể thấy độ trễ khi điện thoại và hệ thống cùng sử dụng mạng Wi-Fi cao hơn so với trường hợp điện thoại sử dụng mạng 5G. Ở điều kiện Wi-Fi, độ trễ trung bình dao động từ 1,75 giây đến 3,23 giây. Trong khi đó, ở điều kiện 5G, độ trễ trung bình giảm xuống khoảng 1,72 giây đến 2,63 giây.

Qua các trường hợp thử nghiệm có thể nhận thấy khả năng tiếp nhận thông báo của hệ thống phụ thuộc vào trạng thái hoạt động của thiết bị và điều kiện kết nối mạng. Để đánh giá mức độ ổn định của cơ chế cảnh báo, nhóm tiến hành tổng hợp kết quả gửi thông báo trong nhiều lần thử nghiệm.

Bảng 4.5. Kết quả khả năng gửi thông báo

Thông số đánh giá	Kết quả
Tổng số lần thử nghiệm	30 lần
Số lần nhận thông báo thành công	25 lần
Tỷ lệ nhận thông báo thành công	83.33%
Trạng thái hoạt động	Tương đối ổn định

Kết quả cho thấy hệ thống có khả năng gửi thông báo ổn định trong các lần thử nghiệm. Tất cả các cảnh báo đều được điện thoại tiếp nhận, tuy nhiên thời gian phản hồi có sự thay đổi tùy theo điều kiện mạng và trạng thái hoạt động của thiết bị.

Trên cơ sở các phản hồi thu được khảo sát từ 15 người, nhóm tiến hành thống kê tỷ lệ đánh giá ở ba mức: Tốt, chấp nhận được và cần cải thiện. Kết quả tổng hợp được trình bày trong bảng 4.6.

Bảng 4.6. Kết quả đánh giá trải nghiệm người dùng

Tiêu chí đánh giá	Tốt	Chấp nhận được	Cần cải thiện
Dashboard dễ quan sát	86,7%	13,3%	0%
Nhận biết màu Xanh/Vàng/Đỏ	100%	0%	0%
Nội dung notification rõ ràng	93,3%	6,7%	0%
Thông số cảm biến dễ đọc	100%	0%	0%

Biểu đồ lịch sử dễ theo dõi	73,3%	20%	6,7%
Thao tác khi có cảnh báo	93,3%	6,7%	0%

Kết quả khảo sát cho thấy ứng dụng Android được người dùng đánh giá tích cực ở hầu hết các tiêu chí. Tiêu chí nhận biết trạng thái thông qua màu Xanh/Vàng/Đỏ đạt tỷ lệ 100%, cho thấy phương pháp mã hóa màu sắc phù hợp với yêu cầu cảnh báo nhanh và giúp người dùng phân biệt rõ các mức an toàn, cảnh báo và nguy hiểm.

Đối với giao diện Dashboard, có 86,7% người dùng đánh giá ở mức tốt. Kết quả này cho thấy cách bố trí thông tin theo từng khu vực giám sát giúp người dùng dễ quan sát trạng thái tổng thể của hệ thống. Bên cạnh đó, tiêu chí nội dung notification rõ ràng đạt 93,3%, chứng tỏ thông báo đã thể hiện được khu vực xảy ra sự cố và mức cảnh báo tương ứng. Tiêu chí thông số cảm biến dễ đọc đạt 100%, cho thấy các giá trị cảm biến được trình bày rõ ràng và thuận tiện cho việc theo dõi.

Tuy nhiên, tiêu chí biểu đồ lịch sử dễ theo dõi có tỷ lệ đánh giá thấp hơn, với 73,3% ở mức tốt, 20% ở mức chấp nhận được và 6,7% cần cải thiện. Nguyên nhân là do biểu đồ hiển thị nhiều thông số cảm biến cùng lúc, khiến người dùng không chuyên cần thêm thời gian để phân biệt từng đại lượng.

Nhìn chung, kết quả đánh giá cho thấy ứng dụng đáp ứng tốt yêu cầu giám sát và cảnh báo từ xa. Giao diện có tính trực quan, màu sắc cảnh báo dễ nhận biết, notification rõ ràng và các thông số cảm biến được hiển thị phù hợp với nhu cầu theo dõi trạng thái hệ thống.

4.4.3. Phân tích và nhận xét

❖ Phân tích khả năng đáp ứng thời gian thực của hệ thống:

Dựa trên kết quả thử nghiệm, hệ thống có độ trễ trung bình 2,47 giây sau 30 lần đo, với độ trễ nhỏ nhất 0,91 giây và độ trễ lớn nhất 4,05 giây. Mức độ trễ này cho thấy hệ thống có khả năng phản hồi trong thời gian ngắn sau khi node ESP32 phát hiện sự cố và kích hoạt cảnh báo tại chỗ.

Đối với mô hình cảnh báo cháy IoT, thời gian phản hồi trung bình dưới 3 giây có thể xem là đáp ứng được yêu cầu cảnh báo gần thời gian thực. Người dùng có thể nhận được notification trong khoảng thời gian ngắn sau khi hệ thống phát hiện bất thường, từ đó có thêm thời gian để kiểm tra khu vực xảy ra sự cố và đưa ra phản ứng kịp thời.

❖ Phân tích các ngoại lệ và nguyên nhân gây độ trễ:

Kết quả đo cho thấy độ trễ không hoàn toàn cố định trong mọi điều kiện thử nghiệm. Một số trường hợp có thời gian phản hồi cao hơn mức trung bình, đặc biệt khi điện thoại ở trạng thái khóa màn hình hoặc ứng dụng chạy nền. Nguyên nhân có thể đến từ cơ chế quản lý ứng dụng nền và tiết kiệm pin của hệ điều hành Android, làm chậm quá trình nhận dữ liệu MQTT hoặc hiển thị notification.

Ngoài ra, chất lượng kết nối mạng tại thời điểm thử nghiệm, số lượng thiết bị truy cập mạng và chu kỳ đọc dữ liệu của ESP32 cũng có thể ảnh hưởng đến thời gian phản hồi. Do phép đo được thực hiện thủ công bằng đồng hồ bấm giờ nên kết quả có thể tồn tại sai số nhỏ. Tuy nhiên, việc đo lặp lại nhiều lần và sử dụng giá trị trung bình giúp hạn chế ảnh hưởng của sai số đơn lẻ.

❖ Nhận xét về thiết kế giao diện và trải nghiệm người dùng:

Giao diện ứng dụng được thiết kế theo mô hình thẻ, phân chia theo từng khu vực như bếp, phòng khách và gara. Cách bố trí này phù hợp với yêu cầu giám sát nhiều vị trí trong hệ thống cảnh báo cháy, giúp người dùng nhanh chóng xác định khu vực đang xảy ra sự cố mà không cần thao tác qua nhiều màn hình.

Việc mã hóa màu sắc theo ba trạng thái Xanh – Vàng – Đỏ hỗ trợ tốt cho quá trình nhận diện mức độ cảnh báo. Màu sắc giúp người dùng phân biệt nhanh trạng thái an toàn, cảnh báo và nguy hiểm, đặc biệt trong tình huống cần phản ứng nhanh.

Tuy nhiên, giao diện vẫn cần được tối ưu thêm cho tình huống khẩn cấp. Khi xảy ra cảnh báo nguy hiểm, ứng dụng nên ưu tiên hiển thị các thông tin quan trọng nhất như khu vực xảy ra sự cố, mức cảnh báo và thời điểm phát hiện. Phần biểu đồ lịch sử cũng cần bố sung chú thích rõ ràng hơn hoặc tách theo từng loại cảm biến để người dùng không chuyên dễ theo dõi.

❖ Nhận xét chung:

Kết quả thử nghiệm cho thấy ứng dụng Android đáp ứng tốt mục tiêu giám sát và cảnh báo từ xa trong phạm vi mô hình. Hệ thống có độ trễ ở mức chấp nhận được, tỷ lệ nhận thông báo thành công đạt 83,33% và giao diện hỗ trợ người dùng nhận biết nhanh trạng thái cảnh báo.

Các hạn chế còn tồn tại chủ yếu liên quan đến trạng thái hoạt động nền của điện thoại, chất lượng kết nối mạng, sai số của phương pháp đo thủ công

và cách trình bày dữ liệu lịch sử. Đây là cơ sở để nhóm tiếp tục cải tiến hệ thống trong các phiên bản tiếp theo.

KẾT LUẬN VÀ ĐỀ NGHỊ

❖ Kết luận

Trong đề tài: “Nghiên cứu, thiết kế ứng dụng giám sát, phân tích dữ liệu và cảnh báo thông minh” nhóm em đã hoàn thành các mục tiêu nghiên cứu đã đề ra. Đề tài đã xây dựng thành công một ứng dụng Android phục vụ giám sát và cảnh báo trong hệ thống phòng cháy chữa cháy dựa trên nền tảng IoT.

Ứng dụng được thiết kế với giao diện trực quan, dễ sử dụng, cho phép hiển thị dữ liệu cảm biến theo thời gian thực. Việc sử dụng giao thức MQTT giúp đảm bảo khả năng truyền dữ liệu nhanh và ổn định giữa các thiết bị và ứng dụng. Và bên cạnh đó, hệ thống đã tích hợp cơ sở dữ liệu Room nhằm lưu trữ dữ liệu lịch sử, hỗ trợ tìm kiếm lịch sử hiển thị dưới dạng biểu đồ và dạng danh sách bản ghi giúp người dùng theo dõi và phân tích sự biến đổi của các thông số môi trường.

Đề tài xây dựng thuật toán cảnh báo kết hợp với các cảm biến nhiệt độ, khói, khí gas và lửa để xác định được trạng thái của hệ thống. Cách tiếp cận này góp phần nâng cao độ chính xác trong phát hiện sự cố và hạn chế báo động giả.

Từ kết quả thử nghiệm và đã đạt được hệ thống có khả năng ứng dụng trong thực tế tại các hộ gia đình, chung cư mini hoặc gara ô tô, giúp người dùng giám sát từ xa và nhận cảnh báo kịp thời, góp phần nâng cao mức độ an toàn cho con người và tài sản.

❖ Đề nghị

Bên cạnh các kết quả đạt được thì hệ thống vẫn còn một số hạn chế. Hiện tại, việc gửi cảnh báo chủ yếu thông qua ứng dụng di động và còn phụ thuộc vào kết nối Internet, do đó có thể bị gián đoạn khi mất mạng. Ngoài ra, hệ thống chưa tích hợp các hình thức cảnh báo khẩn cấp như gửi tin nhắn SMS hay cuộc gọi tự động.

Đề xuất nghiên cứu tiếp theo:

- Tích hợp thêm các phương thức cảnh báo như SMS hay cuộc gọi tự động nhằm đảm bảo thông tin được truyền tải ngay cả khi không có kết nối Internet và nâng cấp hệ thống có thể tự động gửi cảnh báo đến lực lượng cứu hỏa khi gặp sự cố cháy nổ.
- Nghiên cứu áp dụng các kỹ thuật Machine Learning hoặc AI trên dữ liệu lịch sử để dự báo sớm nguy cơ cháy nổ.

TÀI LIỆU THAM KHẢO

Tiếng Việt:

1. Vũ Trung Dũng, Trần Thu Hương, Phạm Anh Tuấn (2022), *Lập trình di động*, Nxb Trường Đại học Kinh tế - Kỹ thuật Công nghiệp, Hà Nội.
2. Lê Hoàng Sơn, Nguyễn Thọ Thông (2020), *Giáo trình lập trình android*, Nxb Xây dựng, Hà Nội.
3. Báo Nhân Dân (2026), “Ứng dụng công nghệ thông minh vào phòng cháy chữa cháy”, *Nghiên cứu và ứng dụng*. <https://nhandan.vn/ung-dung-cong-nghe-thong-minh-vao-phong-chay-chua-chay-post945095.html>. Khai thác ngày 20/03/2026.

Tiếng Anh:

4. Dmitry Jemerov & Svetlana Isakova (2017), *Kotlin in Action*, Manning Publications, Shelter Island. <https://www.manning.com/books/kotlin-in-action>. Khai thác ngày 19/03/2026.

